GenAI using LangChain

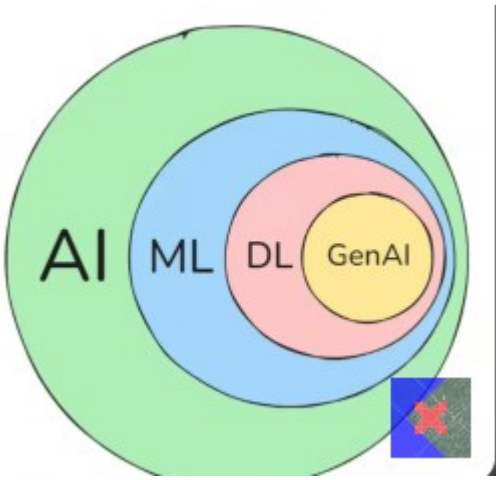
Table of contents:

1. **Generative AI using LangChain**
2. **Introduction to LangChain**
3. **LangChain Components**
4. **LangChain Models**
5. **Prompts in LangChain**
6. **Structured Output in LangChain**
7. **Output Parsers in LangChain**
8. **Chains in LangChain**
9. **What are Runnables in LangChain**
10. **LangChain Runnables-Part 2**
11. **Document Loaders in LangChain**
12. **Text Splitters in LangChain**
13. **Vector Stores in LangChain**
14. **Retrievers in LangChain**
15. **Retrieval Augmented Generation**
16. **Youtube Chatbot using LangChain**
17. **Tools in LangChain**
18. **Tool Calling in LangChain**
19. **Build end-to-end AI Agent in LangChain**

1. Generative AI using LangChain

GenAI is a type of artificial intelligence that creates new content such as text, images, music or code-by learning patterns from existing data, mimicking human creativity.

GenAI impact areas:
1) Customer support
2) Content creation
3) Education
4) Software development



In Generative AI (GenAI), you can either use foundation models or build your own.

- **Foundation Models** are large, pre-trained models like LLMs (Large Language Models) and LMMs (Large Multimodal Models).
    - **LLMs** handle text-based tasks (e.g., GPT, BERT).
    - **LMMs** can process and generate both text and images/videos (e.g., GPT-4 with vision, Gemini, Claude with images).

RISHU KUMAR

You have two main options in GenAI:

1. Using Foundation Models

You can directly use existing pre-trained models via APIs or platforms. These models are already trained on huge datasets and can perform various tasks like:

- Text generation
- Image captioning
- Chatbots
- Document summarization
- Q&A systems

To enhance their performance, you can apply:

- **RAG (Retrieval-Augmented Generation):** Combines foundation models with external knowledge sources (like vector databases) to give more accurate and context-rich responses.
- **Fine-tuning:** You slightly train a model on a specific dataset to adapt it to your custom task.
- **Quantization:** You compress the model to make it run faster and use less memory without much accuracy loss.
- **Vector Databases:** These store and search high-dimensional data (embeddings) efficiently — often used in RAG pipelines.
- **AI Agents:** These are intelligent systems built using foundation models that can reason, plan, and take actions based on goals or prompts. Ex: It will book a train ticket by itself.

2. Building Foundation Models

This means training your own large model from scratch or with transfer learning. It involves:

- **Pretraining:** Training the model on large-scale data (text, images, etc.) to learn general knowledge.
- **Massive compute and data requirements**
- Not typically done by individuals or small teams due to high cost — mostly by companies like OpenAI, Google, Meta, etc.

Summary:

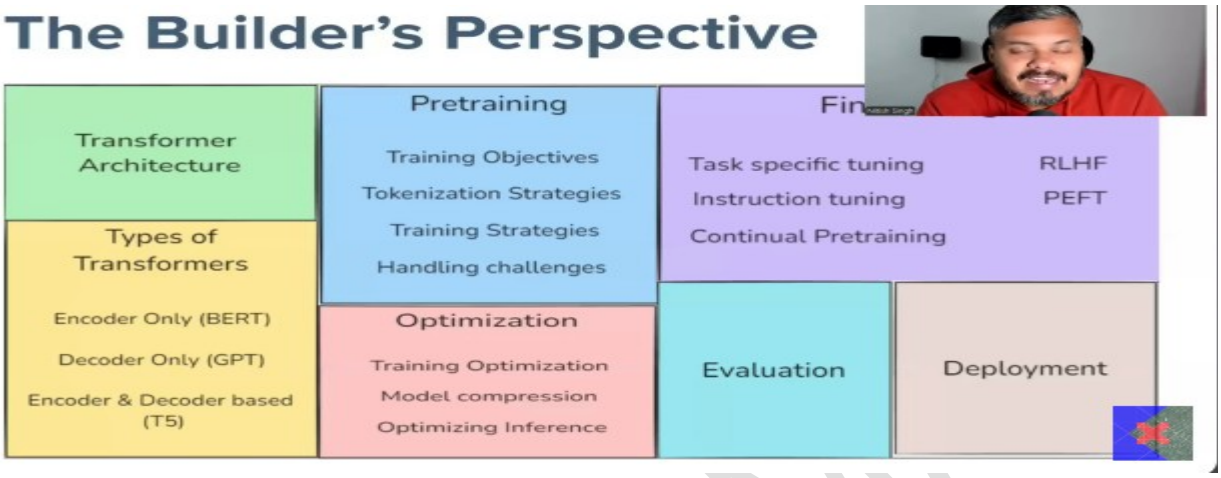
In GenAI, you can:

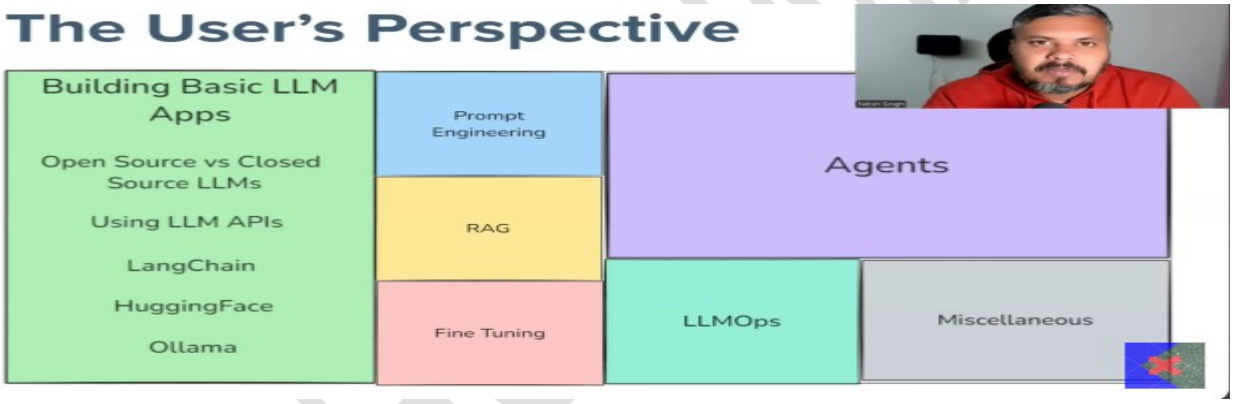
- **Use prebuilt models** (LLMs/LMMs) and enhance them with techniques like RAG, fine-tuning, or quantization.

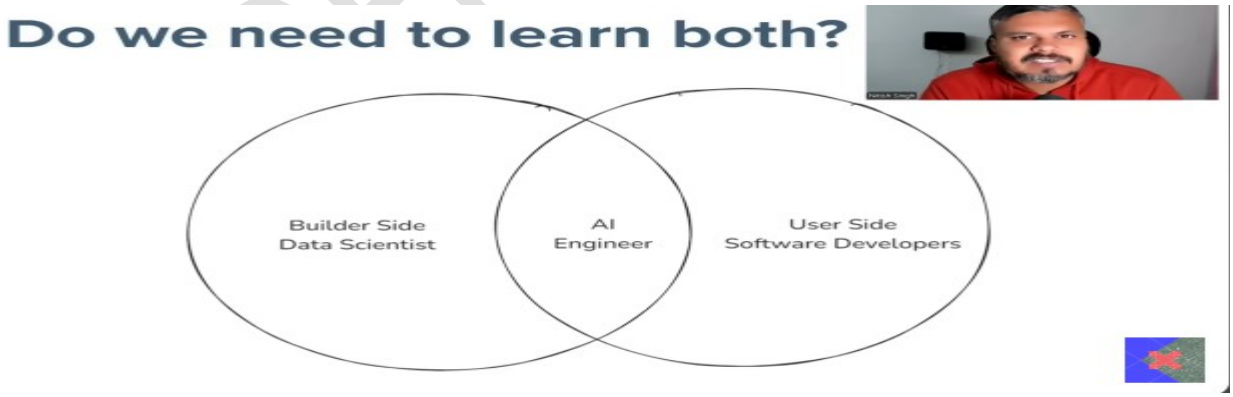
RISHU KUMAR

- **Or build your own foundation models**, which requires large data, compute power, and expertise.

Both paths support creating intelligent applications like chatbots, AI agents, and multimodal tools.



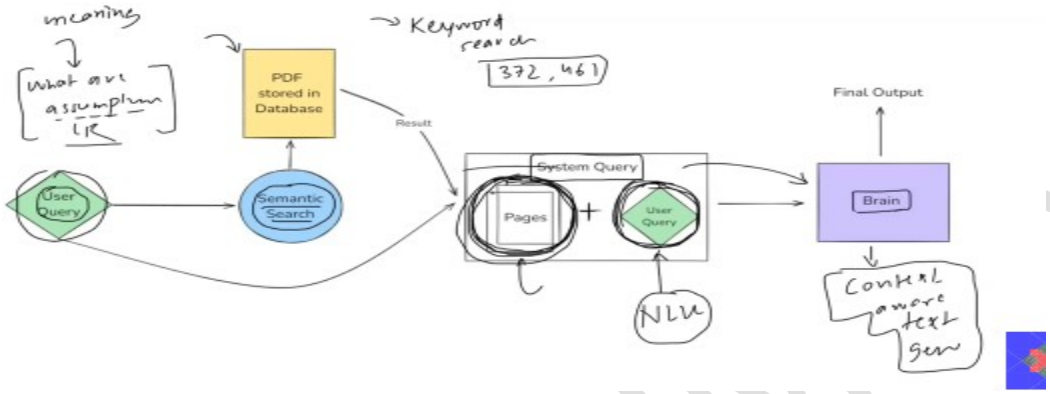




- Langchain supports all open and closed source LLM.
- It gives wholistic view of user's perspective.
- This playlist is based on langchain v3.

2. Introduction to LangChain

LangChain is an open-source framework designed to help developers build applications powered by **Large Language Models (LLMs)** like GPT, Gemini, Claude, etc.



Let's understand with an example:

Suppose you want to build a **reading assistant app**.
A user uploads a **PDF book**, reads it inside the app, and while reading, can ask any question — like clearing doubts, getting summaries, MCQs, etc.

How would the app work?

1. **User uploads a PDF** → PDF is stored in a database.
2. **User asks a question** while reading → e.g., "What does the author mean by X?"
3. We perform **semantic search** on the book to find relevant content.
4. The relevant text + user query is sent to the **"brain"** (LLM) to generate a smart response.

What is Semantic Search and How Does it Work?

Semantic search helps in **understanding meaning**, not just keywords.

Steps:

1. **Split the PDF into chunks** (e.g., by pages or sections using a text splitter).
2. **Convert each chunk into embeddings** (vectors of numbers) using models like BERT, Word2Vec, OpenAI, etc.
3. **Store these embeddings** in a vector database.
4. When the **user asks a question**, convert the question into an embedding too.
5. **Find similar vectors** (text chunks) by comparing with stored embeddings using cosine similarity.

RISHU KUM

6. **Fetch top relevant chunks**, combine with the query, and send to the LLM for a final answer.

How Do We Build the "Brain"?

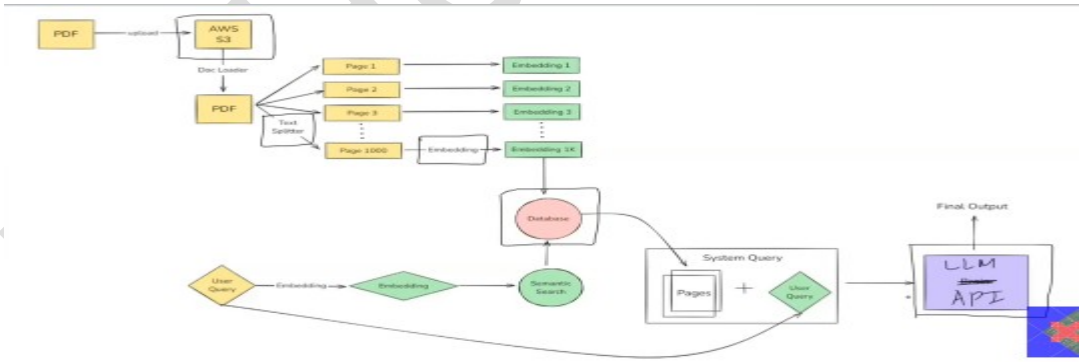
- The **brain** of the system is an **LLM** like GPT or Gemini.
- But LLMs are large and expensive to run on your own servers.
- So, we use **LLM APIs** instead (OpenAI, Gemini, etc.) to keep things efficient and cost-effective.

Challenge: How to integrate all of this?

Connecting:

- File upload
- Text splitting
- Embeddings
- Semantic search
- LLM
- Memory
- State
- Query generation

... manually is complex and error-prone.



That's Where LangChain Helps

LangChain provides ready-to-use modules and pipelines to connect everything easily.

Key Benefits of LangChain:

1. **Chains (Pipelines)**: Connect different steps easily.

   E.g., Load → Split → Embed → Search → Generate Answer.

2. **Model Agnostic**: Use OpenAI, Gemini, Cohere, etc., by changing just 1–2 lines of code.

KUMAR

3. **Ecosystem Integration**:
    - Input from PDF, Excel, websites, cloud, etc.
    - Supports many embedding models and text splitters.
4. **Memory & State Management**: Retain conversation history or previous steps — useful for chatbots.
5. **Fast Prototyping**: You focus on your idea; LangChain handles the plumbing.

What Can You Build with LangChain?

- **Conversational Chatbots**
- **Knowledge Assistants** (e.g., Chatbot trained on a company's documents)
- **AI Agents** (can interact and perform actions — like booking tickets)
- **Workflow Automation**
- **Summarization / Q&A over PDFs or research papers**
- **Company-specific ChatGPT-like tools**

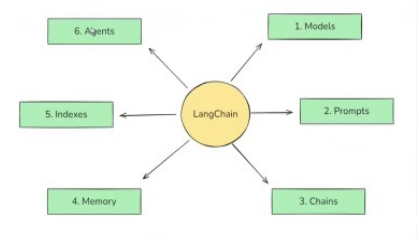
Alternatives to LangChain

Framework	Description	Pros	Cons
LangChain	Modular framework for LLM-powered apps with chaining, memory, and tools	Rich ecosystem, great for agents, supports many LLMs, memory, plugins	Slightly heavier, can feel complex for simple tasks
LlamaIndex	Great for connecting LLMs with **your own data** (documents, DBs, etc.)	Optimized for document-based Q&A, fast, simple APIs	Less focused on agents and chaining; less versatile than LangChain in some workflows
Haystack	Focused on **production-ready NLP/LLM pipelines** (used in enterprises)	Great for search, pipelines, supports RAG well, easy to deploy in real-world apps	Slightly more setup required; more NLP-centric than GenAI-wide

Summary:

- **LangChain** is perfect if you're building a chatbot, knowledge assistant, or AI agent with LLMs.
- It saves you from reinventing the wheel by giving ready-made tools for chaining, memory, embeddings, and model integration.
- **LlamaIndex** is ideal for document-heavy apps with retrieval focus.
- **Haystack** is best for robust NLP pipelines in production.

3. LangChain Components



RISHU KUMAR

1) Models 2) Prompts 3) Chains 4) Memory 5) Indexes
6) Agents

1) Models:
In LangChain, **models** are the core building blocks that allow us to interact with AI systems like **Large Language Models (LLMs)**.

Background: Why Models Are Important?

In traditional **NLP (Natural Language Processing)**, many developers wanted to build chatbots. But there were **two main challenges**:

1. **Understanding the user's question** → called **NLU** (Natural Language Understanding)
2. **Generating a relevant and meaningful answer** → called **context-aware text generation**

How LLMs Solved This

LLMs like **GPT** or **Gemini** are trained on huge amounts of internet data. Because of this, they can:

- Understand natural language (solve NLU)
- Generate meaningful and context-aware answers (solve generation)

But there’s a catch...

New Problem: LLMs are Huge!

- These models have **billions of parameters**, so they are **too large to run locally** on a regular computer.
- To solve this, companies provide access to LLMs via **APIs** (e.g., OpenAI API, Gemini API). You send a prompt, they send back the answer.

Implementation Problem

But what if you're using **multiple APIs or different models** (OpenAI, HuggingFace, Gemini)?
Writing different code for each model becomes messy.

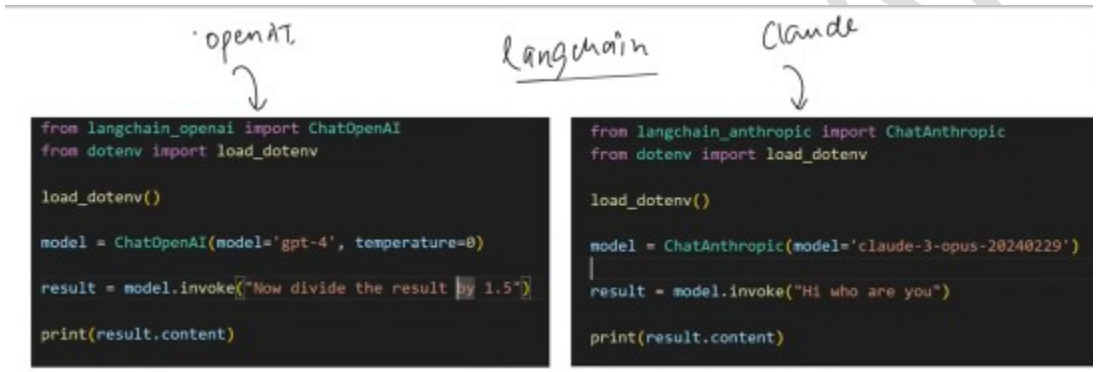
This is where **LangChain helps**.

What Does LangChain Do?

LangChain provides a **common interface** for all models.
You don’t need to worry about the specific code of each LLM — LangChain handles it for you.

RISHU KUMAR

Langchain code:



Two Main Types of Models in LangChain:

1. **Language Models**
   - These are **LLMs** that take **text as input** and give **text as output**.
   - Example use: Chatbots, summarization, text generation.
   - Examples: GPT-4, Claude, Gemini.

2. **Embedding Models**
   - These take **text as input** and give **a vector (list of numbers)** as output.
   - Used for **semantic search** — they help find meaning-based matches, not just keyword-based.
   - Examples: OpenAI Embeddings, HuggingFace Sentence Transformers.

Summary:

- In LangChain, **Models** are interfaces to work with AI — either to generate text or to convert text into vectors.
- **Language Models** → Input: text → Output: text

- **Embedding Models** → Input: text → Output: vector
- LangChain makes it easy to switch between different models/APIs with minimal code changes.

https://python.langchain.com/docs/integrations/chat/

https://python.langchain.com/docs/integrations/text_embedding/

2) Prompts:
A **prompt** is the **input you give to an LLM** (like GPT) to get a response.

Just like asking a smart person a question — the better you ask, the better answer you get.

Why Prompts Matter?

- The **quality of the prompt** directly affects the **quality of the response**.
- Carefully designed prompts guide the model to behave in the way you want.

Types of Prompts in LangChain:

1. **Dynamic & Reusable Prompts**
   - Prompts that have variables or templates (like `{{question}}`, `{{context}}`)
   - Useful for building general-purpose apps (e.g., Q&A, summarization)
2. **Role-Based Prompts:** You assign the LLM a **role** in the prompt like:

   "You are a helpful translator" or "You are an exam evaluator"

3. **Few-Shot Prompts:** Give a **few examples** in the prompt to guide the model:

   Input → Output
   Input → Output
   New input → ?

1. Dynamic & Reusable Prompts

```
from langchain_core.prompts import PromptTemplate

prompt = PromptTemplate.from_template('Summarize {topic} in {emotion} tone')

print(prompt.format(topic='Cricket', length='fun'))
```

2. Role-Based Prompts

```
# Define the ChatPromptTemplate using from_template
chat_prompt = ChatPromptTemplate.from_template([
    ("system", "Hi you are a experienced {profession}"),
    ("user", "Tell me about {topic}"),
])

# Format the prompt with the variable
formatted_messages = chat_prompt.format_messages(profession="Doctor", topic="Viral Fever")
```

DISHU KUMAR

3. Few Shot Prompting

```python
examples = [
    {"input": "I was charged twice for my subscription this month.", "output": "Billing Issue"},
    {"input": "The app crashes every time I try to log in.", "output": "Technical Problem"},
    {"input": "Can you explain how to upgrade my plan?", "output": "General Inquiry"},
    {"input": "I need a refund for a payment I didn't authorize.", "output": "Billing Issue"},
]
```

```python
# Step 2: Create an example template
example_template = """
Ticket: {input}
Category: {output}
"""
```

```python
# Step 3: Build the few-shot prompt template
few_shot_prompt = FewShotPromptTemplate(
    examples=examples,
    example_prompt=PromptTemplate(input_variables=["input", "output"], template=example_template),
    prefix="Classify the following customer support tickets into one of the categories: 'Billing Issue', 'Technical Problem', or 'General Inquiry'.\n\n",
    suffix="\nTicket: {user_input}\nCategory:",
    input_variables=["user_input"],
)
```

3) Chains:

In LangChain, **Chains** allow you to create **multi-step workflows**, where the **output of one step becomes input for the next**.

Example Use-Case:

Suppose you want to build a tool that takes a **1000-word English article** and gives a **100-word summary in Hindi**.

Your chain would look like:

Input (English text)
→ LLM (translate to Hindi)
→ LLM (summarize in Hindi)

Types of Chains:

1. **Sequential Chain**
    - Step-by-step flow
    - E.g., Translate → Summarize → Analyze
2. **Parallel Chain**
    - Run **multiple LLMs or tasks in parallel**
    - E.g., One LLM generates a report, another makes a summary, another finds keywords — all at once.
3. **Conditional Chain**
    - Logic-based chains based on some condition
    - E.g., If customer feedback is **positive**, say “Thank you”; if **negative**, send an alert to the support team.

RISHU KUMAR

4) Indexes:
Indexes connect your LLM-based app to **external data sources** like:

- PDFs
- Websites
- Databases
- Google Drive files

This is how **RAG (Retrieval-Augmented Generation)** works.

4 Core Components of an Index:

1. **Document Loader**
    - Loads data from PDF, Word, website, etc.
2. **Text Splitter**
    - Breaks long text into smaller chunks for better processing
3. **Embedding Model + Vector Store (Vector DB)**
    - Converts text into numerical vectors
    - Stores and retrieves chunks based on similarity
4. **Retriever**
    - When the user asks something, it fetches relevant text chunks by comparing embeddings.

This enables the LLM to **answer questions from your custom data** (like company policies, books, etc.)

5) Memory:
LLM APIs are **stateless** — they forget previous messages.

Memory allows your application to **remember past interactions**, like a human would.

Types of Memory:

1. **ConversationBufferMemory**
    - Stores the full conversation history (like a running log)
2. **ConversationBufferWindowMemory**
    - Stores only the **last few messages** (like a sliding window)
3. **SummarizerMemory**

RISHU KUMAR

- Stores a **summary** of past conversations to save space

4. **CustomMemory**
    - You can create your own memory logic if needed

Useful for building **chatbots** that can hold real conversations over time.

6) Agents:
Agents are **LLMs with extra abilities**:

- They can **reason** and **decide what tool or step to use**
- They can interact with tools like:
    - Web search
    - Python code
    - Calculator
    - APIs
    - Databases

It's like giving the LLM a **brain + hands** — so it can **think and act**

Difference Between Chatbot, GenAI, and Agentic AI (in simple terms):

Type	What it Does	Example
Chatbot	Only **talks** based on your input	ChatGPT answering questions
GenAI App	**Talks + understands** custom data or tasks	PDF Q&A bot, summarizer
Agentic AI	**Thinks + decides + acts** using tools	Books a train ticket, sends an email, fetches real-time data

Summary

LangChain helps you build LLM-powered applications by giving you:

Concept	Purpose
Models	Interface to LLMs and embedding tools
Prompts	Inputs to guide model behavior
Chains	Workflows to connect steps

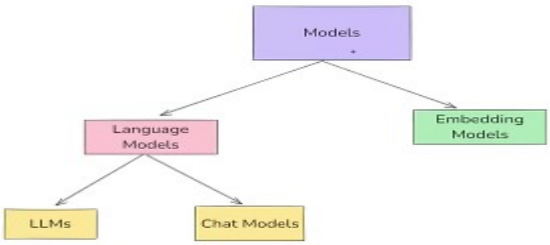
RISHU KUMAR

Concept	Purpose
Indexes	Connect LLM with your data (PDFs, websites)
Memory	Maintain context across sessions
Agents	Let LLMs use tools and take actions

4. LangChain Models

The models component is a crucial part of the framework, designed to facilitate interactions with various language models and embedding models.
It abstracts the complexity of working directly with different LLMs, chat models and embedding models, providing a uniform interface to communicate with them. This makes it easier to build applications that rely on AI-generated text, text embeddings for similarity search, and RAG.

Language models are AI systems designed to process, generate and understand natural language text.



LLMs: General purpose models that is used for raw text generation. They take a string or plain text as input and returns a string or plain text. These are traditionally older models and are not used much now.
Chat Models: Language models that are specifically for conversational tasks. They take a sequence of messages as inputs and return chat messages as output (as opposed to using plain text). They are traditionally newer models and used more in comparison to the LLMs.

Temperature is a parameter that controls the randomness of a language model's output. It affects how creative or deterministic the response is.

Feature	LLMs (Base Models)	Chat Models (Instruction-Tuned)
Purpose	Free-form text generation	Optimized for multi-turn conversations
Training Data	General text corpora (books, articles)	Fine-tuned on chat datasets (dialogues, user-assistant conversations)
Memory & Context	No built-in memory	Supports structured conversation history
Role Awareness	No understanding of "user" and "assistant" roles	Understands "system", "user", and "assistant" roles
Example Models	GPT-3, Llama-2-7B, Mistral-7B, OPT-1.3B	GPT-4, GPT-3.5-turbo, Llama-2-Chat, Mistral-Instruct, Claude
Use Cases	Text generation, summarization, translation, creative writing, code generation	Conversational AI, chatbots, virtual assistants, customer support, AI tutors

Lower values(0-0.5) – more deterministic and predictable.
Higher values(0.7-1.5) – more random, creative and diverse.
For factual answers (codes, math) : 0-0.3
For general QA, explanation: 0.5-0.7
For creative work like story, joke: 0.9-1.2
For more randomness: 1.5+
If **temperature=0**, then everytime we will get the same output for same input.

Open-source language models are freely available models that can be downloaded, modified, fine-tuned and deployed without restrictions from a central provider. Unlike closed-source models such as OpenAI's GPT-4, Anthropic's claude or google's gemini, open-source models allow full control and customization. Ex: Llama, Mistral, Falcon, BLOOM

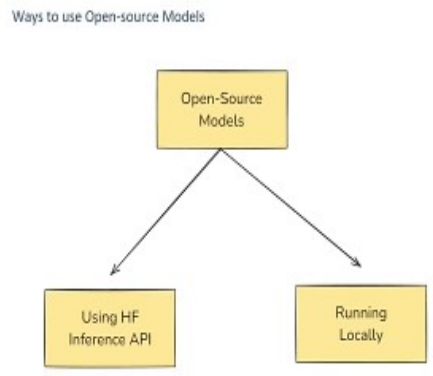
Feature	Open-Source Models	Closed-Source Models
Cost	Free to use (no API costs)	Paid API usage (e.g., OpenAI charges per token)
Control	Can modify, fine-tune, and deploy anywhere	Locked to provider's infrastructure
Data Privacy	Runs locally (no data sent to external servers)	Sends queries to provider's servers
Customization	Can fine-tune on specific datasets	No access to fine-tuning in most cases
Deployment	Can be deployed on on-premise servers or cloud	Must use vendor's API

Some Famous Open Source Models

Model	Developer	Parameters	Best Use Case
LLaMA-2-7B/13B/70B	Meta AI	7B - 70B	General-purpose text generation
Mixtral-8x7B	Mistral AI	8x7B (MoE)	Efficient & fast responses
Mistral-7B	Mistral AI	7B	Best small-scale model (outperforms LLaMA-2-13B)
Falcon-7B/40B	TII UAE	7B - 40B	High-speed inference
BLOOM-176B	BigScience	176B	Multilingual text generation
GPT-J-6B	EleutherAI	6B	Lightweight and efficient
GPT-NeoX-20B	EleutherAI	20B	Large-scale applications
StableLM	Stability AI	3B - 7B	Compact models for chatbots

RISHU KUMAR

We can install these models or use these models from huggingface(largest repository of open-source LLMs).



Disadvantages of open-source models:
- High hardware requirements
- Setup complexity
- Lack of RLHF
- Limited multimodal abilities

5. Prompts in LangChain

Prompts are input instructions or queries that we give to a model (like a chatbot or image generator) to guide its output. Prompts help the model understand **what kind of response** we want — such as an answer, a summary, a story, an image, etc.

Types of Prompts:

1. **Textual Prompts:**
   - These are plain text inputs.
   - Example:
     "Explain quantum physics in simple terms."
2. **Multimodal Prompts:**
   - These involve **multiple input types** like **text + image**, **text + audio**, or even **video**.
   - Example:
     Uploading an image of a dog and asking: *"What breed is this?"*

Static vs Dynamic Prompts:

Type	Description	User Control	Example Use Case
Static Prompt	The prompt is fixed and manually written by the user.	**More control**	Asking: *"Write a poem about rain."*

RISHU KUMAR

Type	Description	User Control	Example Use Case
Dynamic Prompt	The prompt is generated or updated automatically based on some logic or context.	**Less direct control**	A chatbot pulling your name or mood to generate a message like: *"Hi Rishu, here's your daily quote!"*

Summary:

- Prompts are how we tell AI what to do.
- They can be **textual** or **multimodal** (image/video/audio).
- **Static prompts** are manually written and offer **precise control**.
- **Dynamic prompts** are **auto-generated** and can adapt based on the user, context, or system behavior.

A **PromptTemplate** in langchain is a structured way to create prompts dynamically by inserting variables into a predefined template. Instead of hardcoding prompts, **PromptTemplate** allows you to define placeholders that can be filled in at runtime with different prompts. This makes it reusable, flexible and easy to manage, especially when working with dynamic user inputs or automated workflows.

Why use PromptTemplate over f strings?

1. Default validation
2. Reusable
3. LangChain Ecosystem

Types of Messages in Chat-Based AI Systems

In chat-based interactions with language models, there are **three main types of messages** used to structure a conversation:

1. System Message

- **Purpose**: Sets the **overall behavior or personality** of the AI assistant.
- **Who writes it?** The developer or user (behind the scenes).
- **Effect**: It guides the tone, role, and boundaries of the AI's responses.

Example:

"You are a helpful and friendly English teacher who explains things in simple terms."

2. Human Message

- **Purpose**: Represents the **user's input** — a question, instruction, or message to the AI.
- **Who writes it?** The actual user.

Example:

RISHU KUMAR

"Can you explain the difference between a list and a tuple in Python?"

3. AI Message 😊

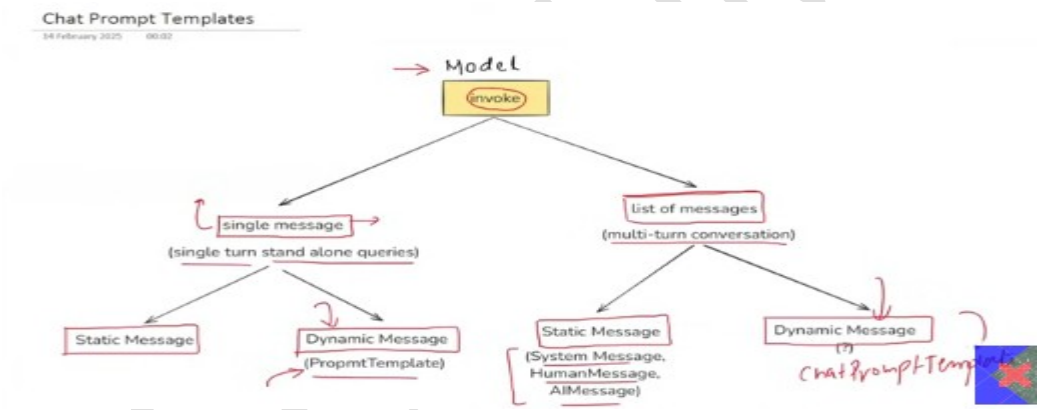
- **Purpose**: This is the **response generated by the AI** based on the system + human message.
- **Who writes it?** The language model (AI).

Example:

"Sure! In Python, a list is mutable (you can change it), while a tuple is immutable (you can't change it after creation)."

Summary Table:

Message Type	Who Sends It?	Purpose
System	Developer / System	Sets the behavior/personality of the AI
Human	User	Asks a question or gives instructions
AI	Model (LLM)	Responds to the human's message



PromptTemplate is used for single turn dynamic messages while **ChatPromptTemplate** is used for multi-turn conversation messages.
In LangChain, when interacting with language models, we use templates to format and structure the prompts. There are two main types based on the use case:

1. PromptTemplate (Single-Turn Prompt)

- Used when you want to send a **single-turn message** to the model (one question → one response).
- It helps you create **dynamic prompts** by inserting variables into a fixed template.

2. ChatPromptTemplate (Multi-Turn Conversation)

- Used when building a **chat-like conversation** where multiple message types (System, Human, AI) are involved.

- Helps structure prompts for **multi-turn dialogs**, like a real chat.

Example Components:

- System Message: Defines AI behavior.
- Human Message: The user's input.
- AI Message: The model's past responses (optional).
- **MessagesPlaceholder**: Used to insert chat history dynamically.

What is MessagesPlaceholder?

- MessagesPlaceholder is a **special component** in ChatPromptTemplate.
- It acts as a **slot** where previous chat history or message list will be inserted **at runtime**.
- Very useful when maintaining a conversation where memory (previous turns) matters.
- A **MessagesPlaceholder** in langchain is a special placeholder used inside a **ChatPromptTemplate** to dynamically insert chat history or a list of messages at runtime. It is used to retrieve and store the chat history.

At runtime, you provide the chat history using the chat_history variable.

Summary

Feature	PromptTemplate	ChatPromptTemplate
Use Case	Single-turn questions	Multi-turn conversations
Message Format	Plain text format	Structured messages (System/Human/AI)
Supports Chat History?	No	Yes (using MessagesPlaceholder)
Dynamic Variables?	Yes	Yes

6. Structured Output in LangChain

Structured output means getting responses from a language model in a **fixed, well-defined format** (like a Python dictionary or JSON), rather than just plain text.
This makes it **easier to parse, validate, and use** the model's output in real-world applications.

In contrast, **textual output** (also called unstructured output) is just free-form text that needs further parsing if you want to extract specific data from it.

Why Use Structured Output?

Some common **use cases** of structured output:

- **Data Extraction** (like extracting sentiment, summary, pros/cons from a review)
- **Building APIs** (where consistent output structure is needed)

RISHU KUMAR

- **AI Agents** (which rely on consistent input/output formats to function properly)

How to Get Structured Output from LLMs?

There are **two types of LLMs** based on their ability to give structured output:

1. **LLMs that support structured output**
   Use: with_structured_output(), use models like OpenAI, claude for this
   These models can directly return output in a format you specify (like a typed dictionary or JSON).
   Steps:
   - First, define your **data format/schema** using TypedDict, Pydantic, or JSON Schema.
   - Then, call .with_structured_output(schema) on the model.
   - Finally, use model.invoke(prompt) and it will return the result in structured format.

   Example behind the scenes:
   LangChain internally generates a **system prompt** like:
   "You are an AI assistant that extracts structured insights from text. Given a product review, extract the following fields: summary, sentiment, pros, cons... Return the result in JSON format."

   ⚠ Note: This method **does not validate** the output format or values. It only follows your instruction.

2. **LLMs that don't support structured output natively**
   Use: **Output parsers**
   These models return normal (unstructured) text, but you can **use a parser** to convert it into a structured format (like JSON).

How to Enforce Structure & Validation? → Use Pydantic

If you want not just structured, but also **validated and type-safe output**, use **Pydantic**:

- Pydantic is a Python library for data validation and parsing.
- You can:
  - Set **data types** and default values
  - Add **constraints** (e.g., max length, regex)
  - Add **descriptions** to fields
- If the model gives incorrect or incomplete data, Pydantic will raise validation errors.

RISHU KUMAR

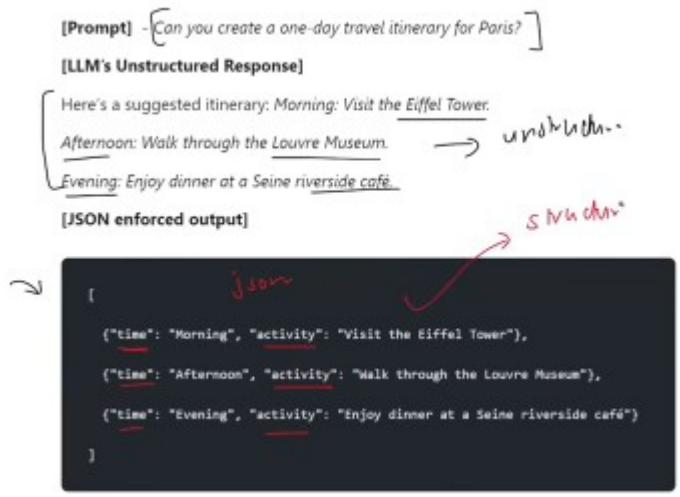
Summary

Type	Method	Validation	Use Case
Structured output (native)	with_structured_output()	No validation	Direct JSON-like output
Structured output (custom)	Use output parsers	With Pydantic	Validate & structure manually

Use TypedDict if:
- only need type hints
- don’t need validation
- trust the LLM to return correct data

Use Pydantic if:
- need data validation
- need default values if the LLM misses fields
- want automatic type conversion

Use JSON schema if:
- don’t want to import extra python libraries
- need validation but don’t need python objects
- want to define structure in a standard json format.



Structured Output in LangChain – When to Use with_structured_output() vs Output Parsers

To get structured output (like JSON) from a Language Model (LLM), we use different methods based on the model's capabilities.

Using with_structured_output()

The with_structured_output() method helps you tell the model exactly **what format** you want the response in (e.g., a dictionary with specific fields). This works well only for models that support **structured output natively**.

There are **two modes** you can set in with_structured_output():

1. **JSON Mode**:
    - Tells the model to return the output in JSON format.
    - Best used with models like **Claude**, **Gemini**, or some **OpenAI models**.
    - Use this when you want readable JSON output directly.
2. **Function Calling Mode**:
    - Allows OpenAI models (like gpt-4) to return output by simulating a function call.

RISHU K

- You define a function (or schema), and the model returns structured data as if it’s calling that function.
- Mainly works with **OpenAI models**.

When Models Don't Support with_structured_output()

Models like **TinyLlama** or **Mistral** (from Hugging Face) **don’t support** structured output via with_structured_output() in either mode (JSON or function calling). If you try it, you'll get an error like:

NotImplementedError: Pydantic schema is not supported for function calling

What to Use Instead?

In such cases, you should use **Output Parsers**:

- You instruct the model to output text in a certain format.
- Then, use a parser (like a regex or custom parser) to convert the text into structured data.
- You can still use Pydantic for validation after parsing.

Summary:

LLM Type	JSON Mode	Function Calling	Output Parser
OpenAI (GPT-4, GPT-3.5)	Yes	Yes	Optional
Claude (Anthropic)	Yes	No	Optional
Gemini (Google)	Yes	No	Optional
HuggingFace (Mistral, TinyLlama)	No	No	Yes (Must Use)

RISHU KUMAR

7. Output Parsers in LangChain

Output parsers in LangChain help **convert raw text output from language models (LLMs)** into a **structured format** like a Python dictionary, JSON, CSV, list, or even a validated object using Pydantic.

You can use them:

- With **any LLM** – whether it supports with_structured_output() or not.
- To make your outputs cleaner, consistent, and easier to use in code.

Why Use Output Parsers?

- Convert free text into structured data.
- Validate the content format.
- Automate parsing without writing custom code every time.

Commonly Used Output Parsers in LangChain

1. StrOutputParser

- **Purpose**: Returns the LLM's output as a plain string.
- **Use Case**: When you just want to use the raw response without any structure.
- **Best For**: Chaining two LLM calls (output of one is input to the next).
- **Note**: You directly get the result as a string — no .content needed.

```
from langchain.output_parsers import StrOutputParser
parser = StrOutputParser()
result = parser.parse("Hello World")  # Output: "Hello World"
```

2. JsonOutputParser

- **Purpose**: Parses the LLM output into a Python dictionary (JSON format).
- **Limitation**: It does **not enforce a schema**. The model can return any JSON structure.
- **Use Case**: When you expect any JSON data but don't care about the structure.

```
from langchain.output_parsers import JsonOutputParser
parser = JsonOutputParser()
parser.parse('{"name": "Rishu", "age": 23}')
```

3. StructuredOutputParser

- **Purpose**: Tells the model to return **structured JSON output** using a **predefined schema**.
- **How**: You define a list of fields using ResponseSchema.
- **Schema Enforcement**: Yes
- **Data Validation**: No (It won't check data types or correctness.)

```
from langchain.output_parsers import StructuredOutputParser, ResponseSchema

fields = [
    ResponseSchema(name="summary", description="Short summary of the content"),
```

RISHU KUMAR

```
    ResponseSchema(name="sentiment", description="Positive, negative, or neutral tone")
]

parser = StructuredOutputParser.from_response_schemas(fields)
```

4. PydanticOutputParser

- **Purpose**: Uses **Pydantic models** to **enforce schema and validate data**.
- **Why Use It**:
  - Type-safe
  - Strong schema enforcement
  - Built-in validation (e.g., required fields, type checks)
  - You can use default values, regex, constraints, etc.

```
from pydantic import BaseModel
from langchain.output_parsers import PydanticOutputParser

class Review(BaseModel):
    summary: str
    sentiment: str

parser = PydanticOutputParser(pydantic_object=Review)
```

5. CsvOutputParser

- **Purpose**: Parses the model output as CSV.
- **Use Case**: When the model generates tabular data and you want each row/column structured.
- **Output**: Usually a list of dictionaries.

6. ListOutputParser

- **Purpose**: Parses the LLM output as a list.
- **Use Case**: When the model outputs items like:

  1. Apple
  2. Banana
  3. Mango

- It removes numbering or bullets and returns ["Apple", "Banana", "Mango"].

RISHU KUMAR

Summary Table

Parser	Output Type	Schema Enforcement	Data Validation	Use Case
StrOutputParser	String	No	No	Raw text or LLM chaining
JsonOutputParser	Dict (JSON)	No	No	General-purpose JSON parsing
StructuredOutputParser	JSON Dict	Yes (Fields only)	No	Basic structured output
PydanticOutputParser	Pydantic Object	Yes	Yes	When you need strict control
CsvOutputParser	List of Dicts	Yes (Headers)	Basic	When expecting table-like output
ListOutputParser	List	No	No	Lists from LLM responses

8. Chains in LangChain

How LangChain Works: Prompting, LLMs, and Chains

Step-by-Step Flow

1. **PromptTemplate**:
   - This is used to create a **customized prompt** by filling in user inputs into a template.
   - For example:

     ```
     from langchain.prompts import PromptTemplate
     prompt = PromptTemplate.from_template("Translate '{text}' to French.")
     final_prompt = prompt.invoke({"text": "Hello"})
     ```

   - final_prompt now becomes: "Translate 'Hello' to French."
2. **LLM Invocation**:
   - Once the prompt is ready, it is passed to the **LLM (Language Model)**.
   - You invoke the LLM with:

     ```
     from langchain.chat_models import ChatOpenAI
     llm = ChatOpenAI()
     result = llm.invoke(final_prompt)
     ```

   - To get the actual result, use:

     ```
     result.content
     ```

RISHU KUMAR

What Are Chains in LangChain?

Chains are used to create **pipelines**, where the **output of one step becomes the input to the next**. This helps in building more complex workflows by combining prompts, LLMs, tools, and logic in a sequence.

Types of Chains

1. Sequential Chain

- Steps are executed one after the other.
- Output from step 1 becomes input for step 2, and so on.
- Best when tasks depend on each other.
- Example: Translate → Summarize → Generate Title

2. Parallel Chain

- Multiple steps run **at the same time**, not one after another.
- Each step gets the **same input**.
- Useful when you want different tasks (e.g., summarization + sentiment analysis) to run independently on the same text.

3. Conditional Chain

- The next step is chosen **based on a condition or output**.
- Like an if-else logic in code.
- Example: If sentiment is positive → generate a compliment, else → give advice.

Summary

Step	Description
PromptTemplate	Creates dynamic prompt using placeholders
invoke()	Fills in the prompt or sends it to LLM
LLM	Language model like GPT, Gemini, Claude, etc.
result.content	Final output text from LLM
Chains	Combines steps into a workflow or pipeline

When building a chain using PromptTemplate, you must **pass the variable name in .invoke() that matches the one defined in the prompt**.

For example:

```
prompt = PromptTemplate.from_template("Translate '{text}' to French.")
```

RISHU KUMAR

Here, the variable name is "text", so when calling invoke, you must do:

final_prompt = prompt.invoke({"text": "Hello"})

If you use a different key (like "input" or "message"), it won't work because LangChain won't know where to insert the value.

Always match the key in .invoke({...}) with the variable name used inside the prompt string.

9. What are Runnables in LangChain

Why Runnables are Important in LangChain:

When ChatGPT was first released in **2022**, followed by the **OpenAI API**, the LangChain team quickly realized something important:

LLMs (Large Language Models) were going to become a key part of future software applications.

So, they asked themselves:
How can we make it easier for developers to build powerful applications using LLMs?

Observations by the LangChain Team

1. **Many companies were building their own LLMs** (OpenAI, Anthropic, Cohere, HuggingFace, etc.).
2. **Different tools and components were being used repeatedly** in every LLM application, such as:
    - Prompt templates
    - Embeddings
    - Document loaders
    - Retrievers
    - Vector stores
    - Output parsers
    - LLMs
3. These components were built **independently** and did **not follow a common standard**.

    Example:

    - Prompt: uses .format()
    - LLM: uses .predict()
    - Parser: uses .parse()

RISHU KUMAR

This inconsistency made it hard to connect components in a clean, scalable way.

Chains: The First Solution

To solve this, LangChain introduced **chains** — a way to link components together.

- Example:
  LLMChain(prompt, llm) would automatically format the prompt, send it to the LLM, and return the result.

They also made higher-level chains like:

- RetrievalQA → combines retriever + LLM to answer questions.
- ConversationalChain → handles chat memory.
- Many others...

But as the number of chains grew, the **codebase became large and hard to maintain**.
Every time a new type of LLM or component came in, it needed special handling.

The Problem

LangChain's components were **not standardized**.
Each one had its own input/output pattern and method names.

This made it difficult to:

- Reuse components easily
- Compose new workflows
- Scale the framework cleanly

The Final Solution: Runnables

To fix this, LangChain introduced a **new abstraction layer** called **Runnables**.

Runnables solve two big problems:

1. **Standardization**: All components (LLM, prompt, parser, retriever, etc.) now follow a **standard .invoke() interface**.
2. **Composition**: You can now **chain, branch, or parallelize** components easily using operators like:
   - | → Pipe output of one component into another
   - RunnableParallel
   - RunnableBranch
   - RunnableLambda

RISHU KUMAR

Example:

prompt | llm | parser

This is much simpler and more readable than the old .predict() or .format() patterns.

Chain Name	Description
LLMChain	Basic chain that calls an LLM with a prompt template.
SequentialChain	Chains multiple LLM calls in a specific sequence.
SimpleSequentialChain	A simplified version of SequentialChain for easier use.
ConversationalRetrievalChain	Handles conversational Q&A with memory and retrieval.
RetrievalQA	Fetches relevant documents and uses an LLM for question-answering.
RouterChain	Directs user queries to different chains based on intent.
MultiPromptChain	Uses different prompts for different user intents dynamically.
HydeChain (Hypothetical Document Embeddings)	Generates hypothetical answers to improve document retrieval.
AgentExecutorChain	Orchestrates different tools and actions dynamically using an agent.
SQLDatabaseChain	Connects to SQL databases and answers natural language queries.

Summary:

- LangChain started by observing repeated patterns in LLM app development.
- They built chains like LLMChain, RetrievalQA, etc., to make life easier.
- But because components were not standardized, it became messy and hard to scale.
- So they created **Runnables**, a **uniform interface** to chain and compose any components.

Now everything (prompt, LLM, parser, retriever, etc.) can be plugged together **like Lego blocks**, making LLM apps easier to build and maintain.

Think of **runnables** like **Lego blocks** — each block does **one job**.

Each Runnable:

- **Takes input** (like a question, text, or document)
- **Does something** (like format it, send to an LLM, or parse the output)
- **Gives output** (like an answer or processed result)

Just like a machine:

input ➡ runnable ➡ output

All Runnables Work the Same Way

They follow a **standard interface**, which means:

RISHU KU

Method	What It Does
.invoke()	Runs on **one input** and gives one output
.batch()	Runs on **multiple inputs** and gives multiple outputs
.stream()	Gives the result in **parts**, useful for streaming responses (like chat)

So no matter what the runnable does (prompting, calling LLM, parsing), you can use these same methods to run it.

You Can Connect Them

You can **chain runnables together**, like this:

Prompt → LLM → OutputParser

Each one takes input, does its job, and passes it to the next.
That whole chain itself becomes a new runnable!

Example in Real Life

Imagine you're making tea:

1. **Boil Water** → input: water → output: hot water
2. **Add Tea Leaves** → input: hot water → output: tea
3. **Add Sugar** → input: tea → output: sweet tea

Each step is a **runnable**, and you can connect them.
The final "Make Tea" workflow is also a runnable.

Why Runnables?

- Makes everything **plug-and-play**
- Easier to build and manage workflows
- All components behave the **same way**
- Clean and readable code

Summary

- Runnables are like **mini building blocks**.
- Each one:
  - Takes input
  - Does a task
  - Gives output
- They all support .invoke(), .batch(), and .stream().

RISHU KUMAR

- You can **chain them** together like Lego to build bigger apps.
- Even the full chain is treated as one big runnable.

Imagine You're in a Kitchen

You want to make a **sandwich**.
You do this step by step:

1. **Take bread**
2. **Spread butter**
3. **Add vegetables**
4. **Put another bread on top**

Each of these steps is doing **one small job** — one **unit of work**.

Now imagine that each of these steps is a **"runnable"**.

Runnable = One Step in a Process

- **Runnable 1:** Take bread
- **Runnable 2:** Spread butter
- **Runnable 3:** Add veggies
- **Runnable 4:** Put the top bread

Each step:

- **Takes input** (like plain bread)
- **Does something** (like spread butter)
- **Gives output** (like buttered bread)

And You Can Join Them

Just like putting steps together to make a sandwich, you can connect these runnables:

Bread ➡ Butter ➡ Veggies ➡ Sandwich

Same in LangChain:

Text ➡ Prompt ➡ LLM ➡ Final Answer

So In Simple Words:

> A **runnable** is just one small step that does one job.
> You can **connect** runnables to do big tasks — like building with Lego blocks.

RISHU KUMAR

Bonus Thought:

If someone gives you a full sandwich, you don't think about the steps. But behind the scenes, all the little **runnables** made it happen.

10. LangChain Runnables-Part 2

Two Types of Runnables in LangChain

1. Task-Specific Runnables

These are the main components that **do a specific job** in an LLM workflow.

Think of them as **tools that perform one task**, like:

- Creating prompts
- Calling an LLM
- Parsing results
- Retrieving documents

These tools have now been **standardized into runnables** so that they can easily fit into a pipeline.

Examples:

- ChatOpenAI: talks to OpenAI's chat model
- PromptTemplate: creates a prompt
- Retriever: fetches relevant documents
- OutputParser: cleans and formats the result

These are called **task-specific runnables** because each does a specific job.

2. Runnable Primitives

These are the **connecting pieces** — they help combine task-specific runnables to build **complete workflows**.

They don't do a job like talking to an LLM, but they **control how tasks are connected and run**, such as:

- Running steps **in order**
- Running steps **in parallel**
- Running something **based on a condition**

You can think of them like **pipes and wires** that connect different machines (task-specific runnables) in the right way.

RISHU KUMAR

Examples:

- RunnableSequence: runs tasks one after another
- RunnableParallel: runs multiple tasks at the same time
- RunnableBranch: runs different steps based on conditions
- RunnableLambda: applies custom logic to the input/output
- RunnablePassthrough: passes data as-is

Simple Analogy:

- **Task-specific runnables** = tools like blender, toaster, microwave
- **Runnable primitives** = the wiring and buttons that control when and how those tools are used

Type	What it Does	Example Components
Task-Specific	Do one job in the pipeline (prompting, LLM, parsing, etc.)	ChatOpenAI, PromptTemplate, Retriever
Runnable Primitives	Connect and control how tasks are executed	RunnableSequence, RunnableBranch, RunnableParallel

Runnable Primitives in LangChain:

LangChain lets you build **AI workflows** by connecting small components called **runnables**. Some of these runnables help with **logic and flow control** — these are called **runnable primitives**.

1. **RunnableSequence**
   - This is used to build a **step-by-step chain**.
   - Each step takes input, does its job, and passes the result to the next step.
   - It's perfect for when your process follows a **strict order**, like:

     Prompt ➜ LLM ➜ OutputParser

   Example: First create a prompt, pass it to an LLM, and then parse the response.

2. **RunnableParallel**
   - Runs **multiple runnables at the same time**, using the **same input**.
   - Each runnable processes the input independently and returns its own result.
   - Outputs are returned as a **dictionary**.

   Use case: You want to send the same question to **two different LLMs** and compare answers.

3. **RunnablePassthrough**
   - A **do-nothing** runnable — just returns the input as output.

RISHU KUMAR

- Useful for testing or when you want to **skip** processing in a step.

Use case: You're debugging and want to pass values unchanged through the chain.

4. RunnableLambda

- Lets you add your own **Python function** inside the workflow.
- You can use it to:
  - Clean or modify input/output
  - Call external APIs
  - Add filters or custom logic

Example: Format the date, clean a string, or convert response to lowercase before passing to next step.

5. RunnableBranch

- Works like an **if-else block**.
- Based on the input, it sends the data to a specific branch (runnable).
- You define **conditions**, and the **first condition that matches** gets executed.
- If none match, it can run a **default branch** (if you set one).

Use case:
If input includes the word "weather", send it to weather-tool.
If it has "news", send it to news-tool.
Else, ask the LLM.

LCEL: LangChain Expression Language

- Writing a full RunnableSequence with many steps can be long and complex.
- So LangChain introduced a **shortcut syntax** called LCEL.
- You use the **pipe symbol (|)** to chain runnables in a sequence.

Example:

```
chain = prompt | llm | output_parser
```

This is the same as writing a full RunnableSequence, but it's **cleaner and simpler**.

Summary Table

Runnable	Purpose	Use Case Example
RunnableSequence	Run steps one after another	Prompt → LLM → Parse

RISHU KUMAR

Runnable	Purpose	Use Case Example
RunnableParallel	Run multiple runnables at once with same input	Compare outputs from multiple models
RunnablePassthrough	Return input as-is	Debugging, placeholder
RunnableLambda	Add custom logic or transformation	Preprocessing, API calls, formatting
RunnableBranch	Conditional logic to route input to different runnables	If-else logic (e.g., weather vs news)
LCEL	('|' syntax)	Simple way to define sequential chains

11. Document Loaders in LangChain

RAG stands for **Retrieval-Augmented Generation**.
It's a technique that combines two steps:

1. **Information Retrieval** → Find relevant documents from a **knowledge base**.
2. **Language Generation** → Use an **LLM (Large Language Model)** to create an answer based on the retrieved documents.

Think of it like this:

- The LLM is your brain.
- The knowledge base is your library.
- If your brain doesn't know something, it quickly checks the library and then answers with the help of that information.

Why use RAG?

RAG has some big advantages over using an LLM alone:

1. **Up-to-date information**
    - LLMs like ChatGPT are trained on past data, so they can't know the latest news or updates.
    - With RAG, you can add a knowledge base containing recent information.
2. **Better privacy**
    - You can store **company data** or **personal data** in a private knowledge base without sending it all to the LLM provider.
3. **No limit on document size**
    - LLMs have token limits, but your knowledge base can be huge. You just retrieve the relevant chunks.

RISHU KUMAR

When does RAG help?

- **Current events** → Example: Today's sports score, latest policy updates.
- **Private data** → Example: Your company's internal reports.
- **Custom knowledge** → Example: A database of recipes or legal cases.

In RAG:

- **Retrieval** = Searching your knowledge base.
- **Generation** = LLM producing an answer using the found information.

Document Loaders in LangChain

Before we can search in our knowledge base, we need to **load documents** into a format LangChain understands.
This is where **Document Loaders** come in.

A **Document Loader**:

- Reads data from different sources (PDFs, text files, websites, CSVs, etc.).
- Converts them into **Document objects** in a **standard format**.

Every **Document object** in LangChain has:

- page_content → The actual text.
- metadata → Information about the document (like file name, page number, source URL).

Examples of Document Loaders

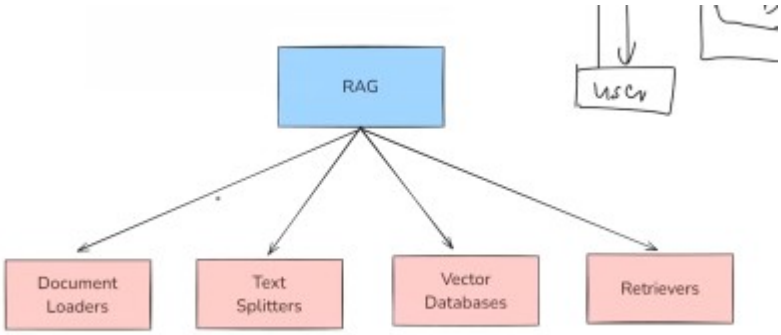
1. **TextLoader**
    - Reads .txt files and turns them into Document objects.
    - **Best for**: Chat logs, scraped text, transcripts, code files.
    - **Limitation**: Only works with .txt files.
2. **PyPDFLoader**
    - Loads text from PDFs, keeping track of pages.
3. **WebBaseLoader**
    - Loads content from websites.
4. **CSVLoader**
    - Loads structured data from CSV files.

In short:
RAG makes LLMs smarter by letting them **look things up** in a knowledge base.

RISHU KUMAR

Document Loaders make sure that whatever data you have (PDF, CSV, TXT, website) is **converted into a common format** so LangChain can process it.

Every Document Loader in LangChain returns a **list of Document objects** as output.



PyPDFLoader

- **What it is:**
  A document loader in LangChain used to read PDF files and convert **each page** into a separate Document object.
- **How it works:**
  It processes the PDF page by page — so if your PDF has **25 pages**, you will get **25 Document objects**.
- **Use case:**
  Helpful when you need to extract and process large amounts of text from PDF files.
- **Limitation:**
  Uses the **pypdf** library under the hood, which may **struggle with scanned PDFs** or **PDFs with complex layouts** (tables, multi-column text, etc.).

Use Case	Recommended Loader
Simple, clean PDFs	`PyPDFLoader`
PDFs with tables/columns	`PDFPlumberLoader`
Scanned/image PDFs	`UnstructuredPDFLoader` or `AmazonTextractPDFLoader`
Need layout and image data	`PyMuPDFLoader`
Want best structure extraction	`UnstructuredPDFLoader`

https://python.langchain.com/docs/concepts/document_loaders/

DirectoryLoader:

- **What it is:**
  A document loader in LangChain that loads **multiple files** from a **folder** (directory).

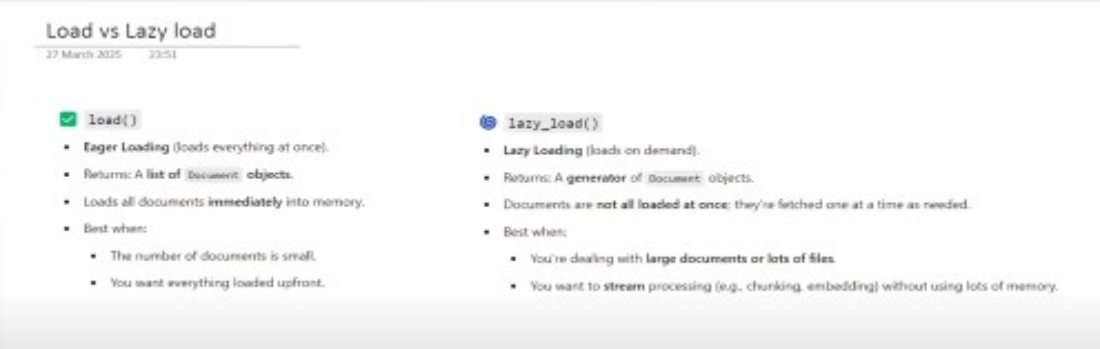
Glob Pattern	What It Loads
`"**/*.txt"`	All `.txt` files in all subfolders
`"*.pdf"`	All `.pdf` files in the root directory
`"data/*.csv"`	All `.csv` files in the `data/` folder
`"**/*"`	All files (any type, all folders)

`**` = recursive search through subfolders

- **How it works:**
  It scans the given folder, applies the appropriate loader for each file type, and returns a **list of Document objects**.
- **When to use:**
  When you have a **collection of files** (e.g., multiple PDFs, text files, or CSVs) stored in one directory and want to load them all at once.

Difference between .load() and .lazy_load()

- **.load()** → Loads **all documents immediately** into memory.
- **.lazy_load()** → Loads documents **one by one (generator)**, which is memory-efficient for very large datasets because it doesn't keep everything in memory at once.



WebBaseLoader

- **What it is:**
  A document loader in LangChain used to **fetch and extract text** from a **webpage (URL)**.
- **How it works:**
  Uses **requests** to get the HTML and **BeautifulSoup** to parse it, extracting only the **visible text** from the page.
- We can pass a list of urls too and we will get that number of document objects.
- **When to use:**
  - Blogs
  - News articles
  - Public websites that are **mostly text-based** and **static**
- **Limitations:**
  - Doesn't handle **JavaScript-heavy pages** well (use SeleniumURLLoader for those).
  - Only loads **static HTML content** (not dynamic elements that load after rendering).

CSVLoader

- **What it is:**
  A document loader in LangChain that loads **CSV files** and converts **each row** into a **separate Document object** (by default).
- **How it works:**
  - Reads the CSV file
  - Creates a Document object for **each row**, where:
    - page_content = text from the row's data
    - metadata = column names or additional file info
- **When to use:**
  - For structured tabular data where each row represents a separate entry or record
  - Examples: product listings, survey results, transaction logs
- **Limitations:**
  - Large CSVs can create **many Document objects**, so .lazy_load() is recommended for memory efficiency.

Custom Document Loader:

If LangChain doesn't have a ready-made document loader for your data source, you can **make your own**.
A **custom document loader** is just a Python class that:

1. Reads your data from **any source** (API, database, special file type, etc.)
2. Converts it into LangChain's **Document** objects, which have:
   - page_content → the actual text you want to use
   - metadata → extra information (e.g., file name, URL, author, timestamp)
3. Returns a **list of these Document objects** so that the rest of LangChain can work with it.

Think of it like this:

- LangChain doesn't care **how** you got the text.
- It only cares that **you give it a list of Document objects** in a standard format.

Technical Steps

LangChain has a base class BaseLoader for custom loaders.
Here's the process:

RISHU KUMAR

1. Import what you need

```
from langchain_core.documents import Document
from langchain_community.document_loaders.base import BaseLoader
```

2. Create your custom loader class

```
class MyCustomLoader(BaseLoader):
    def __init__(self, source):
        self.source = source  # could be file path, API URL, database connection, etc.

    def load(self):
        # Step 1: Read the data
        data = self._read_data()

        # Step 2: Convert data into list of Document objects
        docs = []
        for item in data:
            docs.append(Document(
                page_content=item["content"],    # main text
                metadata={"source": self.source, "extra_info": item.get("info", None)}
            ))
        return docs

    def _read_data(self):
        # Your custom logic goes here
        # Example: reading from a special file format
        with open(self.source, "r", encoding="utf-8") as f:
            lines = f.readlines()
        return [{"content": line.strip()} for line in lines]
```

3. Use your loader

```
loader = MyCustomLoader("my_data.txt")
documents = loader.load()

for doc in documents:
    print(doc.page_content, doc.metadata)
```

Key Points

- You **must return a list** of Document objects (or use .lazy_load() for streaming).
- Each Document needs:
  - page_content → text for processing
  - metadata → optional, but useful for filtering/search later

RISHU KUMAR

- The load() method is **mandatory** in your custom loader.

12. Text Splitters in LangChain

Text splitting is the process of breaking large chunks of text — like articles, PDFs, HTML pages, or books — into smaller, manageable pieces (**chunks**) so that a Large Language Model (LLM) can handle them more effectively.

Why Do We Use Text Splitting?

1. **Overcoming model limitations**
    - LLMs and embedding models have a **maximum input size** (in tokens or characters).
    - Without splitting, large documents would be **truncated** or **rejected**.
    - Splitting ensures we can still process the entire document in smaller parts.
2. **Better downstream task performance**
    - **Embedding**: Shorter chunks → more accurate vector representations.
    - **Semantic search**: Search results become more focused and relevant instead of returning large irrelevant blocks.
    - **Summarization**: Prevents **hallucinations** and **topic drift** (model losing focus).
3. **Optimizing computational resources**
    - Smaller chunks use less memory and can be **processed in parallel**, saving time and cost.

Four Types of Text Splitting in LangChain

1. Length-based Splitting

- Splits text purely based on **size** (characters or tokens).
- **Pros**: Very simple to implement.
- **Cons**:
    - Ignores sentence boundaries, grammar, and meaning.
    - May cut in the middle of a sentence or word.

Chunk Overlap:

- Keeps a certain number of characters from the end of one chunk at the start of the next.
- Maintains **context** between chunks.
- Common practice: overlap = 10–20% of chunk size.
    - Example: If chunk size = 100, overlap = 10–20 characters.

RISHU KUMAR

2. Text Structure-based Splitting

- Respects the **natural structure** of the text.
- Text follows a hierarchy: **paragraphs → sentences → words → characters**.
- Uses **RecursiveCharacterTextSplitter** in LangChain.

How it works:

- First tries to split by **paragraphs** (\n\n).
- If a chunk is still too large, splits by **sentences** (\n).
- If still too large, splits by **words** (spaces).
- Finally, if necessary, splits by **characters**.

Key point:

- Rarely splits in the middle of a word; tries to split at whitespace or punctuation.

3. Document Structure-based Splitting

- Used when documents have a special structure (e.g., **code files**, **markdown files**).
- Not organized in paragraphs/sentences but around **keywords** or **syntax elements**.

Examples:

- Python code → split by \nclass or \ndef.
- Markdown → split by # for headings.

How it works:

- Still uses **RecursiveCharacterTextSplitter** but with **custom separators** specific to the file type.

4. Semantic Meaning-based Splitting

- Splits text based on **meaning** rather than length or syntax.
- Goal: Each chunk should represent a **coherent topic**.
- Example: If a paragraph talks about two different subjects, it gets split into two chunks.

How it works:

- Uses embeddings or similarity scoring to find natural topic breaks.
- Often implemented with a **sliding window approach** (moving window to keep context while splitting).

Summary Table

RISHU KUMAR

Type	Basis for Splitting	Best for	Weakness
Length-based	Fixed size (chars/tokens)	Quick splitting, any text	Breaks sentences, no meaning awareness
Text structure-based	Paragraphs, sentences, words	Articles, essays	May still break meaning
Document structure-based	Keywords, syntax	Code, markdown	Needs predefined separators
Semantic meaning-based	Topics & meaning	Knowledge bases, Q&A	Computationally expensive

13. Vector Stores in LangChain

Suppose we are building a **movie recommendation system**.
We want to recommend movies based on their **description** (storyline, genre, etc.).

Steps:

1. Convert each movie description into an **embedding vector** (a list of numbers that represents meaning).
2. When a user likes movie **M1**, we need to compare its embedding with all other movies' embeddings to find similar ones.
3. But there are challenges:
    - **Storage**: Normal relational databases like MySQL are not designed to store and compare high-dimensional vectors (thousands of numbers per movie).
    - **Speed**: If we have 10 lakh (1 million) movies, comparing M1 with each one directly will be **very slow**.
    - **Semantic Search**: We don't want just keyword matching, but actual *meaning-based* similarity.

Solution → Vector Stores

A **vector store** is a special system/database designed to **store and search embeddings efficiently**.

Key Features of Vector Stores

1. **Storage**
    - Stores vectors (the numerical embeddings) and metadata (like movie name, description, year).
    - Can store **in-memory** (fast but limited) or **on-disk** (scalable for large data).
2. **Similarity Search**

RISHU KUMAR

- Quickly finds the most similar vectors to a query vector.
- Example: Given M1's embedding, find the 5 closest movies.
- Uses **cosine similarity** or other distance measures.

3. **Indexing** (to make it fast)
   - Instead of comparing with all vectors (which is too slow), vector stores use smart techniques:
     - **Approximate Nearest Neighbor (ANN)**: Finds "good enough" matches very fast.
     - **Clustering**: Groups vectors into clusters. First compare with cluster centers, then only search inside the closest cluster. This reduces computations a lot.
4. **CRUD Operations**
   - Just like normal databases, vector stores let you:
     - **C**reate → Add new vectors (new movies)
     - **R**ead → Search for similar vectors
     - **U**pdate → Update movie info or embedding
     - **D**elete → Remove old data

Use Cases of Vector Stores

1. **Semantic Search** → Search based on *meaning* instead of keywords (e.g., searching "space adventure" finds *Interstellar*).
2. **RAG (Retrieval-Augmented Generation)** → LLMs use vector stores to fetch relevant context from documents.
3. **Recommendation Systems** → Like Netflix or Spotify recommending similar movies/songs.
4. **Image / Multimedia Search** → Find similar images, audio, or videos using embeddings.

In short:
A vector store is like a **smart library** for embeddings. Instead of flipping through millions of books one by one, it uses clever indexing and similarity search to instantly find the most relevant ones.

Vector Store vs Vector Database

Vector Store

Vector Store Vs Vector Database

05 April 2025 17:40

- Vector Store
  - Typically refers to a lightweight library or service that focuses on storing vectors (embeddings) and performing similarity search.
  - May not include many traditional database features like transactions, rich query languages, or role-based access control.
  - Ideal for prototyping, smaller-scale applications
  - Examples: FAISS (where you store vectors and can query them by similarity, but you handle persistence and scaling separately).
- Vector Database
  - A full-fledged database system designed to store and query vectors.
  - Offers additional "database-like" features:
    - Distributed architecture for horizontal scaling
    - Durability and persistence (replication, backup/restore)
    - Metadata handling (schemas, filters)
    - Potential for ACID or near-ACID guarantees
    - Authentication/authorization and more advanced security
  - Geared for production environments with significant scaling, large datasets

RISHU KUMAR

- A **storage layer** that stores embeddings (vectors) of documents/texts.
- Used for **similarity search** (e.g., find documents most similar to a query).
- Provides a **common interface** in LangChain with methods like:
    - from_documents → Create a vector store from document objects.
    - from_texts → Create from plain text.
    - add_documents → Add new docs later.
    - add_texts → Add new texts later.
    - similarity_search → Find docs similar to a query.
- Examples in LangChain: **FAISS, Chroma, Pinecone, Qdrant, Weaviate, etc.**

Think of a vector store as a **smart library shelf** where embeddings are stored and retrieved based on similarity.

Vector Database

- A **vector database = vector store + database features.**
- In addition to storing vectors, it provides:
    - **Clustering & indexing** (for faster search at scale).
    - **Scalability** (handle millions of vectors).
    - **Security** (access control, authentication).
    - **Metadata filtering** (search docs with conditions like author, date, tags).
- Examples: **Pinecone, Milvus, Qdrant, Weaviate.**

Think of a vector database as a **full-fledged library system**: not only shelves (vector store) but also indexing, search catalogs, permissions, and filtering.

⚡ **All vector databases are vector stores, but not all vector stores are full vector databases.**

Vector Stores in LangChain

- LangChain integrates with many vector stores, so you can swap one for another with minimal code changes.
- They support **metadata handling** (e.g., storing timestamps, authors, categories with each doc).
- This enables **filter-based retrieval** like “Find similar docs written after 2023 by Author X.”

Chroma Vector Store

- **Lightweight, open-source vector database** (great for local dev or small-to-medium production).

RISHU KUMAR

- Easy to set up and use with LangChain.
- **Hierarchy in Chroma:**
    - **Tenant** → A user (can have multiple databases).
    - **Database** → Contains multiple **collections**.
    - **Collection (like a table)** → Stores multiple **documents**.
    - **Document** → Contains:
        - Embedding vector
        - Metadata (extra info like title, author, timestamp)

You can imagine it like:
Tenant (User) → Database → Collection (Table) → Document (Vector + Metadata).

In short:

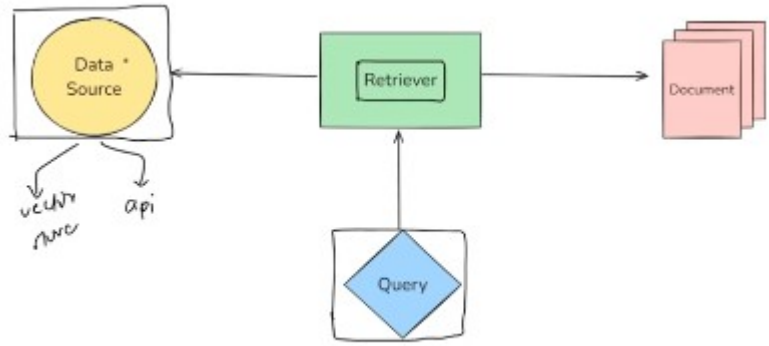
- Use a **vector store** for basic similarity search and embeddings.
- Use a **vector database** if you need scalability, filtering, clustering, and production-grade features.
- **Chroma** is simple, free, and great for local projects, while Pinecone/Qdrant/Milvus are better for large-scale production.

14. Retrievers in LangChain

A **Retriever** is a component in LangChain that **fetches the most relevant documents** from a data source when a user asks a query.
Instead of searching the whole dataset manually, retrievers smartly bring back the **most useful pieces of information**.

All retrievers in LangChain are **Runnables**, which means they can be easily plugged into **chains** or combined with other components.

Types of Retrievers:



1. **Based on Data Source**
    - **Wikipedia Retriever** → fetches info directly from Wikipedia.
    - **Vector Store Retriever** → retrieves docs from vector stores like FAISS, Pinecone, Chroma, etc.

RISHU KUMAR

    - **Arxiv Retriever** → fetches research papers from the Arxiv database.

2. **Based on Search Strategy**
    - **MMR (Maximum Marginal Relevance Retriever)** → balances relevance + diversity to avoid repetitive answers.
    - **Multi-Query Retriever** → generates multiple versions of the query to capture different perspectives.
    - **Contextual Compression Retriever** → compresses documents (e.g., summarization) before returning them, making results more concise.

In short:

- **Retriever = smart document fetcher**.
- **Different retrievers exist depending on where the data comes from and how the search is done**.

https://python.langchain.com/docs/integrations/retrievers/

A **Retriever** is a LangChain component that **fetches the most relevant documents** based on a user's query.
It is **different from loaders** → loaders bring *all documents*, while retrievers bring *only the relevant ones*.

All retrievers are **Runnables**, so you can chain them or plug them into workflows.

Types of Retrievers

1. Based on Data Source

WikipediaRetriever

- Fetches info directly from **Wikipedia API**.
- **How it works**:
    - You give a query → it sends the query to Wikipedia API.
    - Retrieves the most relevant articles.
    - Returns them as **LangChain Document objects**.
- ⚡ Note: It looks like a loader, but it's not → it fetches only what's needed at query time.
- **Limitation**: Keyword-based, not semantic.

Vector Store Retriever (Most Common)

- Works with **vector stores** like FAISS, Chroma, Pinecone.

RISHU KUMAR

- **How it works**:
    1. Each document is converted into an **embedding vector**.
    2. Stored inside the vector store.
    3. Query → also converted into a vector.
    4. Retriever compares query vector with stored vectors → fetches **Top-k most similar docs**.
- **Why not just use similarity_search of vector store?**
    1. Direct vector search = only one retrieval strategy.
    2. Retrievers = let you **try different strategies** like MMR, multi-query, compression, etc.

⚡ Difference: Chroma() vs Chroma.from_documents()

- Chroma() → connect to an existing Chroma database.
- Chroma.from_documents() → creates a **new Chroma collection** from given documents directly.

2. Based on Search Strategy

Maximal Marginal Relevance (MMR Retriever)

- **Problem**: Simple similarity search gives results that are very similar (repeated info).
- **Solution**: MMR balances **relevance + diversity**.
- **How it works**:
    1. Pick the most relevant doc first.
    2. Then pick the next doc that is both **relevant and least similar** to already selected docs.
- **Use case**: In RAG, ensures LLM gets **diverse perspectives** and avoids redundancy.

Multi-Query Retriever

- **Problem**: A single query may not capture all possible phrasings.
- **Example**: "How can I stay healthy?" could mean → diet, exercise, stress management.
- **How it works**:
    1. Original query → expanded into multiple rephrasings by an **LLM**.
    2. Each sub-query retrieves documents.
    3. Results combined + deduplicated.
- **Benefit**: Improves recall, finds info even if docs use different words.

RISHU KUMAR

Contextual Compression Retriever

- **Problem**: Retrieved docs may be too long, with mixed relevant + irrelevant info.
- **How it works**:
    1. Normal retriever fetches top-k docs.
    2. Docs + query sent to an **LLM compressor**.
    3. LLM trims irrelevant parts → keeps only relevant context.
- **When to use**:
    1. Long, noisy documents.
    2. Reduce context length for LLM.
    3. Improve accuracy in RAG.

3. Other Special Retrievers

BM25Retriever

- Uses **classical keyword-based search** (BM25 algorithm, common in search engines).
- Good for situations where semantic embeddings are not available or not needed.

ParentDocumentRetriever

- Splits documents into **small chunks for embeddings** but remembers their **parent document**.
- **How it works**:
    - Retrieves small chunks using embeddings.
    - Returns the **larger parent doc** (not just the chunk).
- **Use case**: Preserves context → better for summarization or long docs.

Self-Query Retriever

- Lets you write queries like:

  "Find articles about AI published after 2021 by OpenAI."

- **How it works**:
    - Uses an LLM to translate natural language into a structured **query with filters**.
    - Applies these filters on vector store metadata.
- **Use case**: When metadata filters (author, date, tags) are important.

RISHU KUMAR

ArxivRetriever

- Retrieves **research papers** from the **Arxiv API**.
- Perfect for academic and scientific RAG pipelines.

Ensemble Retriever

- Combines results from **multiple retrievers** (e.g., keyword + vector).
- Results are ranked/merged.
- **Use case**: When you want robustness (both semantic and keyword coverage).

TimeWeightedVectorRetriever

- Vector retriever that also considers **recency**.
- Useful when recent documents are more important than old ones.

MultiVectorRetriever

- Stores **multiple embeddings per document** (e.g., one for summary, one for title, one for full text).
- Improves retrieval quality because query can match on different representations.

Summary

- **Retriever = smart fetcher of relevant docs.**
- **Data Source retrievers** → Wikipedia, Arxiv, Vector Stores.
- **Search Strategy retrievers** → MMR, Multi-query, Compression.
- **Special retrievers** → BM25, ParentDoc, Self-Query, Ensemble, TimeWeighted, MultiVector.

15. Retrieval Augmented Generation

How an LLM (Large Language Model) is trained:

1. Data Collection

- Huge amounts of text data are gathered from books, websites, articles, code, research papers, etc.
- This becomes the **training dataset**.
- Example: GPT models are trained on trillions of tokens (words/pieces of text).

2. Tokenization

- The text is broken into **tokens** (small pieces like words or subwords).

RISHU KUMAR

- Example: "ChatGPT is smart" → ["Chat", "G", "PT", " is", " smart"].

3. Neural Network Architecture (Transformer)

- LLMs use the **Transformer architecture**.
- Key components:
  - **Encoder/Decoder layers** (for processing sequences)
  - **Attention mechanism** (helps the model focus on important words in context).

4. Pretraining (Self-Supervised Learning)

- The model is trained to **predict the next token** in a sentence.
- Example: Input: "The cat is on the" → Model should predict "mat".
- This is done **billions of times** across all text data.
- The model learns **grammar, facts, reasoning, style** from patterns in data.

5. Optimization

- Training uses **backpropagation** and **gradient descent**.
- Loss function (usually cross-entropy) measures how wrong the prediction is.
- The model updates weights to improve predictions.

6. Fine-tuning

- After pretraining, the model can be fine-tuned on specific data:
  - **Instruction tuning** → making the model follow instructions better.
  - **Domain-specific tuning** → medical, legal, finance data.

7. RLHF (Reinforcement Learning with Human Feedback)

- Humans rate model outputs (good/bad).
- A reward model is trained on these ratings.
- The LLM is fine-tuned using reinforcement learning to prefer **helpful, safe, human-like answers**.
- This is why GPT doesn’t just dump raw internet text but responds conversationally.

In summary:
LLMs are trained in 3 stages:

1. **Pretraining** → Learn language patterns from massive text.
2. **Fine-tuning** → Specialize on instructions or domains.

RISHU KUMAR

3. **RLHF** → Align with human preferences and safety.

How LLMs Store Knowledge

- A **Large Language Model (LLM)** is trained on massive amounts of text data.
- During training, it learns **patterns of language, facts, and reasoning**.
- This knowledge is stored inside its **parameters** (weights and biases).
- Since it stores knowledge in parameters, we call this **parametric knowledge**.

Example: When you ask a question, the model doesn't "look up" the answer in a database. Instead, it generates the answer **step by step (word by word)** by drawing from patterns stored in its parameters.

Limitations of Parametric Knowledge

- **Not real-time** → The model cannot know recent events (e.g., yesterday's news).
- **No private data** → It won't know about your company's documents unless trained or connected with external tools.
- **Hallucinations** → Sometimes it may generate **confident but incorrect answers**.

How to Overcome These Limitations

We can **adapt** or **update** an LLM for specific needs:

1. **Search / RAG (Retrieval-Augmented Generation)** → Combine LLM with external knowledge sources (databases, documents, web).
2. **Fine-tuning** → Retrain the model on domain-specific data so it learns to answer in a certain way.

Analogy

- Think of a **pretrained LLM** as a student who has completed a **B.Tech degree** (general knowledge).
- Fine-tuning is like giving the student **company-specific training** so they can perform **specialized tasks** in that company.

Ways of Fine-Tuning

1. **Supervised Fine-Tuning (SFT)**
    - Train the LLM with **labeled data**: *Prompt → Desired Output*.
    - Example: Fine-tune a general LLM to become a **medical assistant** by training it on medical Q&A.
2. **Continued Pretraining** (Domain Adaptation)

RISHU KUMAR

    - Train on **unlabeled text** from your domain.
    - Example: Feed the model tons of legal documents so it “absorbs” legal language style and concepts.
3. **RLHF (Reinforcement Learning with Human Feedback)**
    - Humans rank multiple model outputs.
    - The model is then trained to prefer outputs humans like.
    - This is how ChatGPT became **helpful, safe, and aligned**.

Process of Supervised Fine-Tuning

1. **Collect data** → Create a domain-specific dataset (prompt → output).
2. **Choose tuning method**:
    - **Full parameter tuning** → retrain all parameters (very expensive).
    - **Parameter-efficient methods** → like **LoRA, QLoRA** (only train small parts of the model).
3. **Train for a few epochs** on the dataset.
4. **Evaluate & Safety test**:
    - Exact match accuracy
    - Hallucination rate
    - Safety/ethical checks

Challenges with Fine-Tuning

- **Computationally expensive** → Big models need GPUs and lots of time.
- **Technical expertise required** → Not beginner-friendly.
- **Not flexible for dynamic data** → If private data changes often (e.g., product catalog, stock data), you’d need to **re-fine-tune again and again**.
    - In such cases, **RAG (retrieval + LLM)** is often more practical.

In short:
LLMs learn general world knowledge during pretraining, stored in parameters. Fine-tuning adapts them to specific domains (like company training). However, fine-tuning is costly and not great for fast-changing data—so retrieval-based methods are often better.

How LLMs Solve Their Limitations

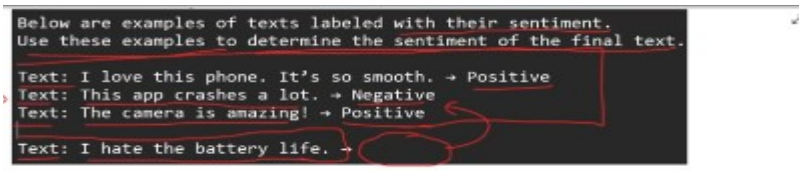
LLMs face three main challenges:

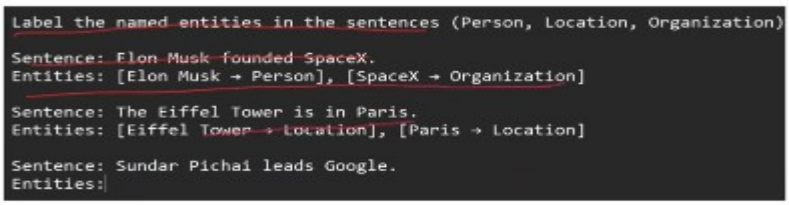
RISHU KUMAR

- **Knowledge cutoff** (they only know what they were trained on).
- **Factuality issues** (hallucinations).
- **Context length limits** (can't remember everything).

Two major approaches address these:

1. In-Context Learning (ICL)





- Instead of retraining or fine-tuning the model, we *teach it on the fly* by providing examples or instructions **within the prompt**.
- Example:

  Q: What is 2+2?
  A: 4

  Q: What is 3+5?
  A: 8

  Q: What is 10+7?
  A:

- The model learns the pattern in the *prompt itself*.

Properties:

- Called **few-shot prompting** when we give examples.
- **Zero-shot prompting** when we only provide instructions.
- An **emergent property** of very large models like GPT-3 (175B). The paper *"Language Models are Few-Shot Learners"* showed that scale made this ability appear, even though the models weren't explicitly trained for it.

2. Retrieval-Augmented Generation (RAG)

- Instead of only relying on the model's frozen parameters, we **inject external knowledge dynamically**.
- Pipeline:

  User Query + Retrieved Context ---> Prompt to LLM ---> Response

- Example: In a video-learning chatbot, the transcript of the current video chunk is retrieved and passed as context.
- The LLM then blends **parametric knowledge** (stored in weights) with **retrieved knowledge** (external, fresh, domain-specific).

RISHU KUMA

This makes the model:

- More factual (since it grounds answers in retrieved docs).
- More up-to-date (since new docs can be indexed anytime).
- Flexible (different contexts can be plugged in per query).

In short:

- **ICL** = model learns patterns from examples in the *prompt*.
- **RAG** = model uses external knowledge retrieved at runtime.

RAG is a method that combines **Information Retrieval** + **Text Generation**.
It helps Large Language Models (LLMs) give more accurate, up-to-date, and reliable answers by using an **external knowledge base** in addition to the model's pre-trained knowledge.

RAG works in **4 main steps**:

1. Indexing (Building the External Knowledge Base)

Indexing means preparing your knowledge base so that it can be **efficiently searched** later when a query comes in.

It consists of **4 sub-steps**:

1. **Document Ingestion**
    - Load your source data into memory (PDFs, Word docs, web pages, databases, etc.).
    - Tools: LangChain loaders like PyPDFLoader, CSVLoader, TextLoader, etc.
2. **Text Chunking**
    - Since documents are usually long, we split them into smaller, meaningful pieces (called *chunks*).
    - This ensures retrieval works well, because embeddings handle short text better than very long ones.
    - Tools: RecursiveCharacterTextSplitter, SemanticChunker, etc.
3. **Embedding Generation**
    - Each chunk is converted into a **dense vector (embedding)** that captures its meaning in numerical form.
    - These embeddings allow similarity search (finding semantically close chunks).
    - Tools: OpenAIEmbeddings, SentenceTransformerEmbeddings, etc.
4. **Store in a Vector Database**

RISHU KUMAR

- Both the embeddings and their original text (plus metadata like source, page number) are stored in a **vector database**.
- This database becomes the **external knowledge base** for retrieval.
- Tools: Local: FAISS, Chroma; Cloud: Pinecone, Weaviate, Qdrant

After this step, we have a searchable external knowledge base.

2. Retrieval (Finding Relevant Chunks at Query Time)

When the user asks a question, we need to search the external knowledge base to find the most **relevant pieces of text**.

Steps:

1. **Query Embedding**
   - Convert the user's query into an embedding using the *same embedding model* used during indexing (so dimensions match).
2. **Semantic Search**
   - Compare the query embedding with all stored embeddings in the vector database.
   - The system retrieves the top chunks that are most semantically similar.
3. **Ranking & Filtering**
   - The retrieved chunks are ranked by similarity.
   - Irrelevant or less useful ones are filtered out.
4. **Fetch Results**
   - The top chunks (usually top 3–5) are returned with their original text and metadata.

This ensures the system picks only the **most useful parts of your knowledge base** for answering the question.

3. Augmentation (Building the Prompt)

Now we combine the **user's query** with the **retrieved chunks** into a single enriched prompt.

- Example:

  User query: "What are the benefits of drinking green tea?"
  Retrieved context: "Green tea contains antioxidants like catechins, helps in fat loss, improves brain function..."
  Augmented Prompt → "Answer the user's question using this context: Green tea contains antioxidants...
  Question: What are the benefits of drinking green tea?"

RISHU KUMAR

This ensures the LLM is not only relying on its pre-trained memory but also uses **up-to-date, factual, and domain-specific context**.

4. Generation (LLM Creates the Final Answer)

Finally, the **LLM generates a response** based on:

- The **user's question**, and
- The **retrieved + augmented context**

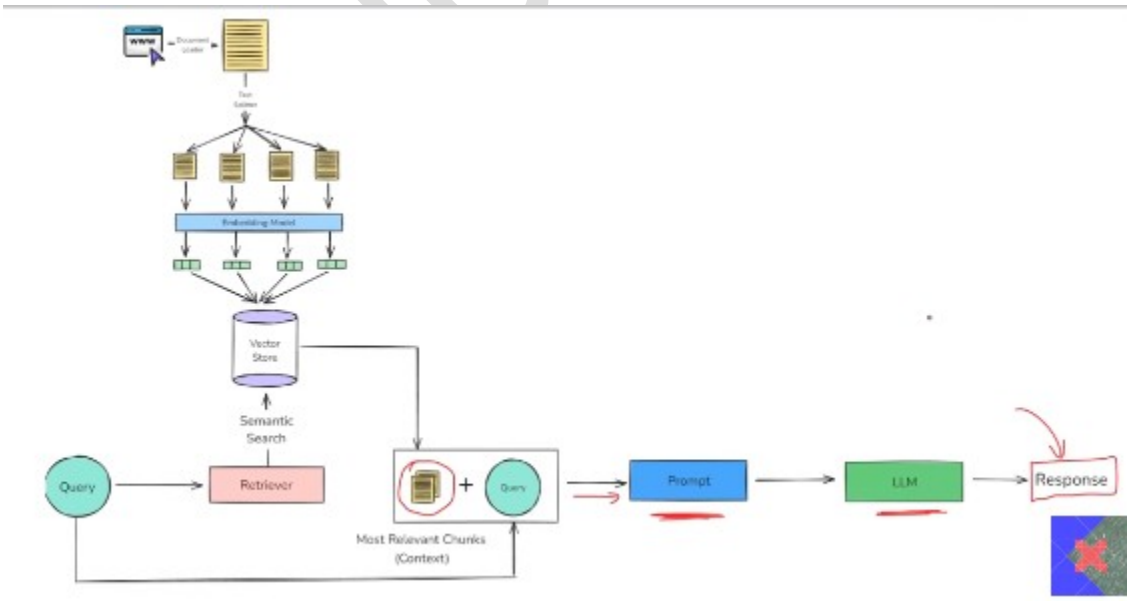
Since the model is now grounded with **external knowledge**, the output is:

- More accurate
- Less hallucinated
- Context-aware

Summary (Simple Flow)

- **Indexing** → Prepare knowledge base (ingest → chunk → embed → store)
- **Retrieval** → At query time, search knowledge base to get relevant chunks
- **Augmentation** → Merge user query + retrieved chunks into enriched prompt
- **Generation** → LLM generates the final answer using the prompt

This is why RAG is powerful: It combines **LLM reasoning** + **real-world factual knowledge** without retraining the model.



RAG is a technique that combines **information retrieval** and **text generation** to make Large Language Models (LLMs) more accurate and reliable.

JI KUMAR

Instead of relying only on the LLM's internal knowledge (which might be outdated or limited), RAG lets the model **fetch external knowledge** at query time and then use it to generate answers.

RAG works in **4 main steps**:

1) Indexing (Building the external knowledge base)

Indexing is the process of preparing an external knowledge base so that it can be **efficiently searched later**.
It consists of 4 sub-steps:

1. **Document Ingestion**
    - Load your data (e.g., PDFs, text files, websites, databases) into memory.
    - Tools: LangChain loaders like PyPDFLoader, WebBaseLoader, etc.
2. **Text Chunking**
    - Break large documents into **smaller, meaningful text chunks** so the LLM can process them effectively.
    - Example: Splitting a 200-page PDF into small paragraphs.
    - Tools: RecursiveCharacterTextSplitter, SemanticChunker.
3. **Embedding Generation**
    - Convert each chunk into a **dense vector (embedding)** that captures the meaning of the text.
    - Embeddings are numerical representations of text.
    - Tools: OpenAIEmbeddings, SentenceTransformerEmbeddings.
4. **Store into a Vector Store**
    - Save the embeddings along with their original text and metadata in a **vector database**.
    - This acts as the **external knowledge base**.
    - Tools:
        - Local: FAISS, Chroma
        - Cloud: Pinecone, Qdrant

End result of indexing: You now have a searchable **vector database** that links text chunks with their embeddings.

2) Retrieval (Finding relevant knowledge in real-time)

When a user asks a question, the system needs to find **relevant pieces of information** from the vector store.

RISHU KUMAR

Steps:

1. Generate an **embedding for the query** using the same embedding model (so both query and text chunks are in the same vector space).
2. Perform a **semantic search** in the vector store to find chunks most similar in meaning to the query.
3. Rank and order the results by relevance.
4. Fetch the **top-k relevant text chunks** (e.g., top 3 or top 5).

End result of retrieval: You get the most useful text chunks from your knowledge base related to the query.

3) Augmentation (Creating an enriched prompt)

- In this step, the **retrieved chunks** are combined with the **user's original query** to form a **new, enriched prompt**.
- Example:
    - User's question: *"What is RAG in NLP?"*
    - Retrieved context: definitions and explanations from your vector store.
    - Augmented prompt: *"The user asked: 'What is RAG in NLP?' Based on the following documents: [retrieved chunks] … provide a clear explanation."*

End result of augmentation: The LLM now has both the **query** and **relevant external knowledge** in its input.

4) Generation (LLM creates the final answer)

- Finally, the LLM uses its **text generation ability** to create a coherent answer.
- Since the model now has access to both the **user's query** and the **retrieved context**, its output is:
    - More accurate
    - Less likely to hallucinate
    - More grounded in real data

End result of generation: A **final response** that combines the LLM's reasoning with external verified knowledge.

Summary in One Line

RAG = **Indexing** (prepare knowledge) → **Retrieval** (find context) → **Augmentation** (combine with query) → **Generation** (LLM answers).

RISHU KUMAR

When we say **RAG is cheaper than fine-tuning**, the idea is:

- **Fine-tuning**: You take a base LLM and actually **train it** again with your domain-specific data. Training means heavy GPU usage, high cost, and repeated updates whenever your knowledge base changes.
- **RAG**: You don't retrain the model. Instead, you **train/prepare only the knowledge base (indexing)** — e.g., embedding documents and storing them in a vector database. Once this is done, the LLM itself is **frozen** (no new training). You only retrieve and augment at runtime.

So technically, there **is some "training-like" work in RAG** (embedding generation for documents, building the index), but it's **much lighter and one-time** compared to fine-tuning the LLM itself.

That's why people say **RAG doesn't involve training** — they mean **no model fine-tuning** is required. Instead, it's just **embedding + indexing**, which is cheap and fast.

16. Youtube Chatbot using LangChain

Improvements in a RAG Application

1. UI-Based Enhancements

- Make the app more **user-friendly & interactive**.
- Add features like:
    - **Chat-like interface** (instead of plain Q&A).
    - **Document upload** (PDF, TXT, DOCX).
    - **Highlighting retrieved context** (so user sees *why* answer is given).
    - **Feedback buttons** (thumbs up/down to collect user ratings).
    - **Source citations** (showing from which document the answer was taken).

2. Evaluation

Using tools like **Ragas** or **LangSmith** for systematic evaluation:

- **Context Recall** → Is the retrieved context actually relevant to the query?
- **Faithfulness** → Is the answer factually correct and grounded in the context (not hallucinated)?
- **Answer Relevancy** → Does the answer directly answer the user's question?

Continuous evaluation helps catch weaknesses and tune retrieval/prompting.

3. Indexing

How documents are processed and stored:

RISHU KUMAR

- **Document Ingestion** → Ability to handle multiple formats (PDF, CSV, website scraping, APIs).
- **Text Splitting** → Split docs into **chunks** (too big → misses info, too small → loses meaning). Smart splitting (like recursive character splitter) is key.
- **Vector Store** → Choosing the right database (FAISS, Pinecone, Weaviate, Chroma, Milvus).

4. Retrieval Improvements

Break it into **3 stages**:

Pre-Retrieval (improving the query before search)

- **Query Rewriting with LLM** → Turn vague user queries into more precise ones.
- **Multi-Query Generation** → Generate different phrasings of the query to retrieve more coverage.
- **Domain-Aware Routing** → If system has multiple specialized knowledge bases (legal, finance, medical), send query to the right one.

During Retrieval

- **MMR (Maximal Marginal Relevance)** → Balances *relevance* and *diversity* in retrieved docs.
- **Hybrid Retrieval** → Combine dense (vector) + sparse (keyword/BM25) retrieval for better recall.
- **Reranking** → Use a small model (e.g., cross-encoder) to rank the top results for higher accuracy.

Post-Retrieval

- **Contextual Compression** → Summarize or filter out irrelevant parts of retrieved context so LLM sees only useful info (saves tokens + improves answers).

5. Augmentation

- **Prompt Engineering**: carefully structured instructions for consistent output.
- **Answer Grounding**: enforce responses tied strictly to retrieved documents.
- **Context Window Optimization**: smart selection of top chunks within LLM's context length limit.

6. Generation Enhancements

- **Answer with Citation**: show sources for transparency & trust.
- **Guardrailing**: enforce safety, structure, and avoid sensitive or irrelevant outputs (via tools like Guardrails AI).
- **Structured Output**: JSON, tables, graphs when needed instead of plain text.

RISHU KUMAR

7. System Design Innovations

- **Multimodal RAG** → support text + image + audio + video documents.
- **Agentic RAG** → use LLM agents to plan, retrieve, and reason in multiple steps (Toolformer / LangChain Agents).
- **Memory-based RAG** → long-term memory for user history across sessions, not just per query.

If you apply these systematically, your RAG system becomes:

- **More accurate** (better retrieval + reranking).
- **More trustworthy** (faithfulness + citations).
- **More usable** (UI + feedback loops).

17. Tools in LangChain

https://python.langchain.com/docs/integrations/tools/

Agents in LangChain (Simplified Explanation)

What are tools?

- A **tool** is basically a **Python function or an API** that we wrap in a way so that an LLM (like GPT) can understand and call it whenever needed.
- Example:
    - A tool could be a function to **fetch today's weather**,
    - or a tool to **search Google**,
    - or a tool to **query a database**.

What LLMs can do (and can't do)?

- LLMs are very good at:
    1. **Reasoning** (thinking logically)
    2. **Language generation** (explaining or writing answers)
- But LLMs cannot directly:
    1. Access live data (weather, news, stock prices)
    2. Do reliable math or calculations
    3. Call APIs
    4. Run Python code
    5. Interact with databases

RISHU KUMAR

Think of an LLM like a **human who can think and talk, but has no hands or legs**.
Tools are like giving that human **hands and legs**, so they can actually **take action** in the real world.

Types of Tools

1. **Built-in tools** – Already provided by LangChain (like search, calculator, etc.)
2. **Custom tools** – You write your own functions/APIs and register them as tools.

How tools fit into the Agent ecosystem

- An **AI Agent** = **LLM + Tools + Decision-making loop**
- In simple words:
  An agent is an **LLM-powered system that can think (reason), decide what to do, and then take action** using external tools or APIs to achieve a goal.
- Example:
  Suppose you ask: *"What's the weather in Pune right now, and should I carry an umbrella?"*
    - The agent uses the **reasoning ability of the LLM** to understand the task.
    - Then it decides: "I need to use the **Weather API tool** to get live data."
    - Calls the tool → gets weather info.
    - Finally, the LLM **generates a natural language answer** like:
      "It's raining in Pune right now, so yes, you should carry an umbrella."

So, in short:

- **LLM = brain (reason + speak)**
- **Tools = hands & legs (do things)**
- **Agent = full system (brain + body) that can autonomously solve tasks**

Built-in Tools in LangChain:

- **What they are**:
  Built-in tools are **ready-made tools provided by LangChain**. They come with all the logic already implemented, so you don't need to write any extra code.
- **Why they're useful**:
    - Saves time → you don't reinvent the wheel.
    - Reliable → they're tested and optimized by LangChain.
    - Easy to use → usually just import, configure (if needed), and run.
- **Examples of built-in tools**:
    - **RequestsGetTool** → fetch data from the web.

RISHU KUMAR

    - **PythonREPLTool** → execute Python code.
    - **AIPluginTool** → call APIs that follow the OpenAI plugin standard.
    - **WikipediaQueryRun** → get information from Wikipedia.
- **How you use them**:
  You simply import the tool, maybe give it some configuration (like an API key), and then call it like any other tool.

In short:
A built-in tool is like a **ready-to-use app on your phone** → you don't need to code it yourself, you just install and use it.

DuckDuckGoSearchRun	Web search via DuckDuckGo
WikipediaQueryRun	Wikipedia summary
PythonREPLTool	Run raw Python code
ShellTool	Run shell commands
RequestsGetTool	Make HTTP GET requests
GmailSendMessageTool	Send emails via Gmail
SlackSendMessageTool	Post message to Slack
SQLDatabaseQueryTool	Run SQL queries

In **LangChain**, both **tools** and other components (like retrievers, chains, LLMs) are designed around the **Runnable interface**.

That means:

- Every tool is just a wrapper around a **Python function or API**.
- Since it's a Runnable, it provides the same standard methods:
    - .invoke(input) → run the tool on a single input
    - .batch(inputs) → run it on multiple inputs at once
    - .stream(input) → stream back results

So when you define a **custom tool**, it's not just something the agent can call—it also behaves like any other **LangChain runnable**, making it easy to plug into pipelines or workflows.

Custom Tools in LangChain

A **custom tool** is a tool that **you define yourself**. Unlike built-in tools, which come ready-made, custom tools let you create logic specific to your project.

Use custom tools when:

- You want to **call your own APIs** (e.g., weather API, stock prices API).
- You want to **encapsulate business logic** (e.g., rules/calculations unique to your app).
- You want the LLM to **interact with your database, product, or application**.

How to Make Custom Tools

There are **multiple ways** to create custom tools in LangChain.
No matter the method, the process follows 3 main steps:

1. **Define a function** → Write your normal Python function.
2. **Add type hints** → Helps LangChain understand input/output format.
3. **Convert into a tool** → Wrap it so the LLM can use it.

When we connect an LLM and a tool, the LLM doesn't directly see the Python function. Instead, it receives a **tool schema in JSON format** that includes:

- **title** → tool name
- **description** → what the tool does
- **properties** → input parameters (name, type, description)
- **required** → which inputs are mandatory

This schema allows the LLM to "decide" when and how to use the tool.

Ways to Create Custom Tools mostly:

1. **Using @tool decorator**
    - The simplest way.
    - Just write a Python function, add type hints, and decorate it with @tool.
2. **Using StructuredTool and Pydantic**
    - Use when your tool needs **structured inputs** (like multiple parameters).
    - Pydantic models help define input/output schema more robustly.
    - A **Structured Tool** in langchain is a special type of tool where the input to the tool follows a structured schema, typically defined using a pydantic model.
3. **Using BaseTool class**
    - The most flexible and advanced method.
    - Allows you to customize how tools behave internally.
    - Good for complex tools (e.g., tools that maintain state or call multiple APIs).
    - BaseTool is the abstract base class for all tools in LangChain, it defines the core structrue and interface that any tool must follow, whether it's a simple one-linear or a fully customized function.
    - All other tool types like @tool, StructuredTool are built on top of BaseTool.

RISHU KUMAR

    - Here we can create async version of the tool when we have to handle concurrency.

In short:

- **Built-in tools** = Ready-made.
- **Custom tools** = Your own functions, wrapped so the LLM can call them.
- Tools expose a **schema (JSON)** so LLMs know how to use them.
- You can create them via **@tool, StructuredTool, or BaseTool** depending on complexity.

Concurrency one of the **biggest reasons to use BaseTool** instead of just @tool or StructuredTool.

- With BaseTool, you can implement both:
    - ._run() → for synchronous execution
    - ._arun() → for asynchronous execution

This gives you **control over concurrency**, which is useful when:

- You’re hitting APIs that support async requests (e.g., fetching multiple results in parallel).
- You want your RAG or agent to **continue reasoning while waiting** on a tool call.
- You’re working in environments like **FastAPI, asyncio apps, or LangServe** where async is needed.

Example:

```
from langchain.tools import BaseTool
from typing import Optional
import aiohttp
import requests

class WeatherTool(BaseTool):
    name = "weather_tool"
    description = "Get the weather for a given city."

    def _run(self, city: str) -> str:
        """Sync version"""
        response = requests.get(f"http://api.weatherapi.com/{city}")
        return response.text

    async def _arun(self, city: str) -> str:
        """Async version"""
        async with aiohttp.ClientSession() as session:
            async with session.get(f"http://api.weatherapi.com/{city}") as resp:
                return await resp.text()
```

Now, if your agent is async-enabled, it will prefer _arun() to keep everything non-blocking.

RISHU KUMAR

In **LangChain**, a **Toolkit** is just a **container (bundle) of related tools** packaged together for **convenience, reuse, and modularity**. Instead of registering each tool individually, you can load a **toolkit** and get a ready-made set. Here it is used when multiple tools are related so we can club them and make a toolkit.

Example: GoogleDriveToolkit

It might contain tools like:

- `GoogleDriveCreateFileTool` : Upload a file
- `GoogleDriveSearchTool` : Search for a file by name/content
- `GoogleDriveReadFileTool` : Read contents of a file

- **UploadFileTool** → upload a file to Google Drive
- **DownloadFileTool** → download a file from Drive
- **SearchFilesTool** → search for files by name/type
- **DeleteFileTool** → delete a file from Drive

So instead of importing/creating each tool manually, you just do:

```
from langchain_community.agent_toolkits import GoogleDriveToolkit

toolkit = GoogleDriveToolkit()
tools = toolkit.get_tools()   # gives you all bundled tools
```

Now your agent can **invoke any Drive-related functionality** without you wiring each tool separately.

Some other common toolkits in LangChain:

- **SQLDatabaseToolkit** → query, insert, update tables in a SQL DB
- **OpenAPIToolkit** → interact with REST APIs via OpenAPI spec
- **VectorStoreToolkit** → search & retrieve documents from a vector DB
- **SlackToolkit** → send/read messages from Slack

So, toolkit = **domain-specific toolbox**

Decorator and Generator:

1. Python Decorator

A **decorator** is just a way to **wrap extra functionality around an existing function** without changing its actual code.

Think of it like putting a gift in a wrapper — the gift is still the same, but the wrapper adds something extra (like making it look fancy).

Example without decorator:

```
def greet():
```

RISHU KUMAR

```
    print("Hello!")

greet()
```

Output:

Hello!

Now, what if we always want to log before greeting?

Using a decorator:

```
def logger(func):   # decorator function
    def wrapper():
        print("Logging: About to call the function...")
        func()   # call the original function
        print("Logging: Function finished.")
    return wrapper

@logger
def greet():   # greet is now wrapped by logger
    print("Hello!")

greet()
```

Output:

```
Logging: About to call the function...
Hello!
Logging: Function finished.
```

Intuition:

- @logger means → “wrap greet inside logger”
- You didn’t change greet’s code, but you **enhanced it**

Real-world use: decorators are used for logging, authentication checks, caching, timing function execution, etc.

RISHU KUMAR

2. Python Generator

A **generator** is a special function that can **pause** and **resume** its execution using the keyword yield.

Think of it like a TV remote — instead of playing the whole movie at once (like a normal function returning a list), a generator **gives you one frame at a time when you ask for it**.

Example without generator:

```
def numbers_list(n):
```

```
    result = []
    for i in range(n):
        result.append(i)
    return result

print(numbers_list(5))
```

Output:

```
[0, 1, 2, 3, 4]
```

⚠ Problem: For very large n (say 1 billion), this will take **huge memory**.

Example with generator:

```
def numbers_generator(n):
    for i in range(n):
        yield i   # instead of returning all at once, yield one by one

gen = numbers_generator(5)

for num in gen:
    print(num)
```

Output:

```
0
1
2
3
4
```

Intuition:

- yield is like **"return, but I'll come back later"**
- Generator **remembers its state** between calls
- Much more **memory efficient** than lists

Real-world use: reading large files line by line, streaming data, infinite sequences (like Fibonacci), etc.

Key Difference in One Line:

- **Decorator** = "wrap extra behavior around a function"
- **Generator** = "produce values one at a time, on demand"

1. **Decorators in Python**

RISHU KUMAR

Intuition

A **decorator** is a **function that takes another function as input, adds some extra behavior, and returns a new function**.
Think of it like:

- You have a plain pizza (original function).
- You add toppings (decorator).
- You still have a pizza, but now with **extra flavor**.

Step 1: Functions are first-class citizens in Python

This means:

- You can pass functions as arguments to other functions.
- You can return functions from other functions.

Example:

```
def say_hello():
    return "Hello!"

def greet(func):
    print("Calling function:", func())

greet(say_hello)
```

Output:

```
Calling function: Hello!
```

Step 2: Writing a basic decorator

```
def decorator(func):   # takes a function
    def wrapper():
        print("Before function call")
        func()   # run original function
        print("After function call")
    return wrapper    # return modified version
```

Now, if we do:

```
@decorator
def say_hi():
    print("Hi!")

say_hi()
```

RISHU KUMAR

Output:

```
Before function call
Hi!
After function call
```

Why use decorators?

1. **Code reuse** – same logging/authentication can be applied to many functions.
2. **Separation of concerns** – keep main function clean, and put side concerns in decorator.

Real-world decorators

- **@staticmethod**, **@classmethod**, **@property** → built-in decorators
- **@app.route()** in Flask → turns a function into a web endpoint
- **@lru_cache** from functools → cache function results

Decorators with Arguments

```
def repeat(n):
    def decorator(func):
        def wrapper(*args, **kwargs):
            for _ in range(n):
                func(*args, **kwargs)
        return wrapper
    return decorator

@repeat(3)
def greet(name):
    print(f"Hello, {name}!")

greet("Rishu")
```

Output:

```
Hello, Rishu!
Hello, Rishu!
Hello, Rishu!
```

2. Generators in Python

Intuition

Normally, a function **runs once and returns** a value.
But a generator function can **pause in the middle (using yield)** and **resume later**, continuing where it left off.

RISHU KUMAR

Think of it like:

- A **movie DVD** — a normal function returns the **whole movie at once**.
- A **TV channel** — a generator streams the **movie scene by scene**, only when you ask for it.

Step 1: Normal function vs Generator

```
def normal_function():
    return [1, 2, 3]

def generator_function():
    yield 1
    yield 2
    yield 3
```

Using:

```
print(normal_function())
gen = generator_function()
print(next(gen))   # 1
print(next(gen))   # 2
print(next(gen))   # 3
```

Output:

```
[1, 2, 3]
1
2
3
```

Notice: next(gen) moves the generator **to the next yield**.

Step 2: Why Generators are powerful

Imagine you want numbers from 1 to **1 billion**.

- With normal list → RAM explodes
- With generator → only one number at a time is in memory.

```
def big_numbers(n):
    for i in range(n):
        yield i

gen = big_numbers(10**9)
print(next(gen))  # 0
print(next(gen))  # 1
```

RISHU KUMAR

Real-world uses

1. **Reading large files**

```
def read_large_file(filename):
    with open(filename, "r") as f:
        for line in f:
            yield line.strip()
```

2. **Streaming data** (API responses, logs, sensors).
3. **Infinite sequences** (like Fibonacci):

```
def fibonacci():
    a, b = 0, 1
    while True:
        yield a
        a, b = b, a+b

gen = fibonacci()
for _ in range(5):
    print(next(gen))
```

Output:

```
0
1
1
2
3
```

Key Differences

Decorator	Generator
Modifies or enhances functions	Produces a sequence of values
Uses @decorator syntax	Uses yield
Focus: wrapping behavior	Focus: memory-efficient iteration
Example: logging, authentication, caching	Example: streaming big data, infinite sequences

In one line:

- **Decorator** → Add extra behavior to a function
- **Generator** → Produce values one at a time on demand ⚡

18. Tool Calling in LangChain

What is Tool Binding?

Tool binding is the process of **connecting tools to an LLM** so that the LLM can actually use them.

RISHU KUMAR

When we "bind" tools, we are basically **teaching the LLM:**

1. **Which tools are available** – so it knows what it can use.
2. **What each tool does** – using descriptions, so it knows when to use which tool.
3. **What input format to follow** – through the schema (the structure of input/output, like JSON).

Think of it like adding apps to your smartphone and giving them proper names & instructions. Without this step, the phone (LLM) wouldn't know the apps (tools) exist or how to use them.

Why is Tool Binding Important?

- If tools are not bound, the LLM is just a **text generator**.
- Once tools are bound, the LLM becomes **actionable** – it can fetch real data, call APIs, search the web, or interact with databases.

Limitation

Not every LLM supports tool binding.
Some advanced ones (like OpenAI's GPT-4 function calling, Gemini, Claude with Tools, LangChain integrations) can bind tools.
Others may not have this capability and will only generate text.

Intuition with Example

Imagine you have an LLM acting as your **personal assistant**.

- Without tool binding → You ask: *"What's the weather in Pune?"* → It just guesses from training data.
- With tool binding → It knows there's a **WeatherAPI tool**, understands the input schema ({"city": "Pune"}), calls the API, and gives you the **real-time weather**.

In short:
Tool binding = registering tools with an LLM so it can see, understand, and use them properly.

What is Tool Calling?

Tool Calling is the step where the **LLM decides**:
"Do I need to use a tool to answer this query?"

If yes, the LLM does **not** run the tool itself (LLMs can't execute code). Instead, it generates a **structured response** that says:

1. **Which tool to use** (tool name)
2. **What inputs to give it** (arguments/schema)

Then the **LangChain framework (or you)** actually runs the tool with those arguments and gives the result back to the LLM.

RISHU KUMAR

Key Point

- The **LLM only "suggests" the tool and inputs**.
- The **execution** of that tool happens **outside the LLM** (by LangChain or your own code).

Example (Intuition)

Imagine you are talking to a smart assistant (LLM) that has access to two tools:

- **Calculator Tool**
- **Weather API Tool**

Case 1:

You: *"How are you?"*
LLM: No tool needed → it just replies: *"I'm fine, thank you!"*

Case 2:

You: *"What's 12 × 9?"*
LLM: Decides a tool is needed. It outputs something like:

```
{
  "tool": "Calculator",
  "arguments": {"expression": "12 * 9"}
}
```

LangChain sees this → runs the **Calculator Tool** → gets 108 → sends it back to the LLM → LLM answers:
"The answer is 108."

Case 3:

You: *"What's the weather in Pune?"*
LLM: Chooses the **Weather Tool** and provides the city name as an argument.

In LangChain

When the LLM suggests a tool, you can check the tool call with something like:

```
tool_calls = response.tool_calls
print(tool_calls[0].name)      # Which tool
print(tool_calls[0].args)      # With what arguments
```

Then you (or LangChain automatically) run that tool.

In short:

- **Tool Binding** = making tools available to the LLM (registering).

RISHU KUMAR

- **Tool Calling** = LLM actually decides *when & how* to use those tools during a conversation.

Tool Execution:

1. Tool Execution (recap)

- **Tool Calling** → LLM only *suggests*: "Call multiply(a=5, b=6)."
- **Tool Execution** → LangChain (or you) actually runs the Python function:

  result = multiply(5, 6) # gives 30

- After execution, LangChain wraps the result in a **ToolMessage** → so the LLM knows: "Yes, the tool ran, and here's the output."

2. What is InjectedToolArg?

Sometimes, a tool requires an input that the **LLM should not (or cannot) provide**.
Instead, **you (developer) or the runtime** must supply it automatically.

This is where **InjectedToolArg** comes in.

- It marks a function argument as **"don't let the LLM fill this, I'll inject it myself."**
- Example: maybe you already computed something earlier (like a conversion_rate) or pulled it from a secure source (like an API key).

By using InjectedToolArg, you stop the LLM from hallucinating/filling wrong values.

Example with InjectedToolArg

```
from langchain_core.tools import tool, InjectedToolArg
from typing_extensions import Annotated

@tool
def convert_currency(
    amount: float,
    conversion_rate: Annotated[float, InjectedToolArg],
) -> float:
    """Converts amount into another currency using a conversion rate."""
    return amount * conversion_rate
```

- Here, conversion_rate is **injected** by you at runtime, not by the LLM.
- When the LLM calls this tool, it will only provide amount.
- LangChain/runtime will fill conversion_rate automatically (say from a DB, API, or previous tool output).

So if LLM says:

RISHU KUMAR

```
{
  "tool": "convert_currency",
  "arguments": {"amount": 100}
}
```

At runtime, you inject: conversion_rate = 83.2
→ Tool runs: convert_currency(amount=100, conversion_rate=83.2)
→ Output: 8320.

3. What is Annotated and why is it used?

Annotated is a typing utility from Python's typing_extensions.
It lets you **attach metadata to type hints**.

In our case:

conversion_rate: Annotated[float, InjectedToolArg]

means:

- This argument is a **float**
- But it also has a special behavior: **InjectedToolArg**

So **Annotated** = "Type hint + Extra info for frameworks."
LangChain uses this extra info to know:

- Which arguments should come from the LLM (normal ones)
- Which arguments should be injected by developer/runtime (those marked with InjectedToolArg)

In short:

- **Tool Execution** → the real function runs.
- **ToolMessage** → returned result after execution.
- **InjectedToolArg** → tells LangChain "don't ask LLM to fill this, I'll inject it myself."
- **Annotated** → allows us to attach this special instruction (InjectedToolArg) to normal Python type hints.

RISHU KUMAR

19. Build end-to-end AI Agent in LangChain

The Manual Way (Without AI)

When you plan a trip on Makemytrip:

1. You open the site.
2. Enter **flight details** (source = Pune, destination = New York, date, class).
3. Book the flight.
4. Then go again, search **hotels in New York**, filter by budget, and book.
5. Then go again, search **cabs** for airport pickup, and book.

Each step, *you* have to click, enter details, and make decisions.

With AI Agent

Now imagine you are chatting with an **AI Travel Agent** in natural language:

> You: "Book me a flight from Pune to New York next Monday, business class. Also arrange a hotel near Times Square, and a cab from JFK airport."

The **Agentic AI** does the following:

1. **Understands your goal** (full trip planning).
2. **Breaks it into sub-tasks**: flight → hotel → cab.
3. **Thinks (reasoning)**: which tools are needed? (Flight booking API, hotel booking API, cab booking API).
4. **Calls tools automatically** with correct inputs (no need for you to enter them manually).
    - Calls flight booking tool with {source=Pune, dest=NYC, date=Monday, class=Business}.
    - Calls hotel booking tool with {location=Times Square, check-in/out dates}.
    - Calls cab booking tool with {pickup=JFK, drop=hotel}.
5. **Combines results** and gives you one answer:

    > "Your flight is booked, Hotel Hilton Times Square is reserved, and cab pickup from JFK is confirmed ✈."

Why is this called Agentic AI?

- Because the AI is not just **answering questions** (like ChatGPT usually does).
- It is **acting like an agent** — meaning:
    - It can **decide what to do next**
    - It can **choose the right tools** (flight API, hotel API, cab API)
    - It can **execute actions** on your behalf

RISHU KUMAR

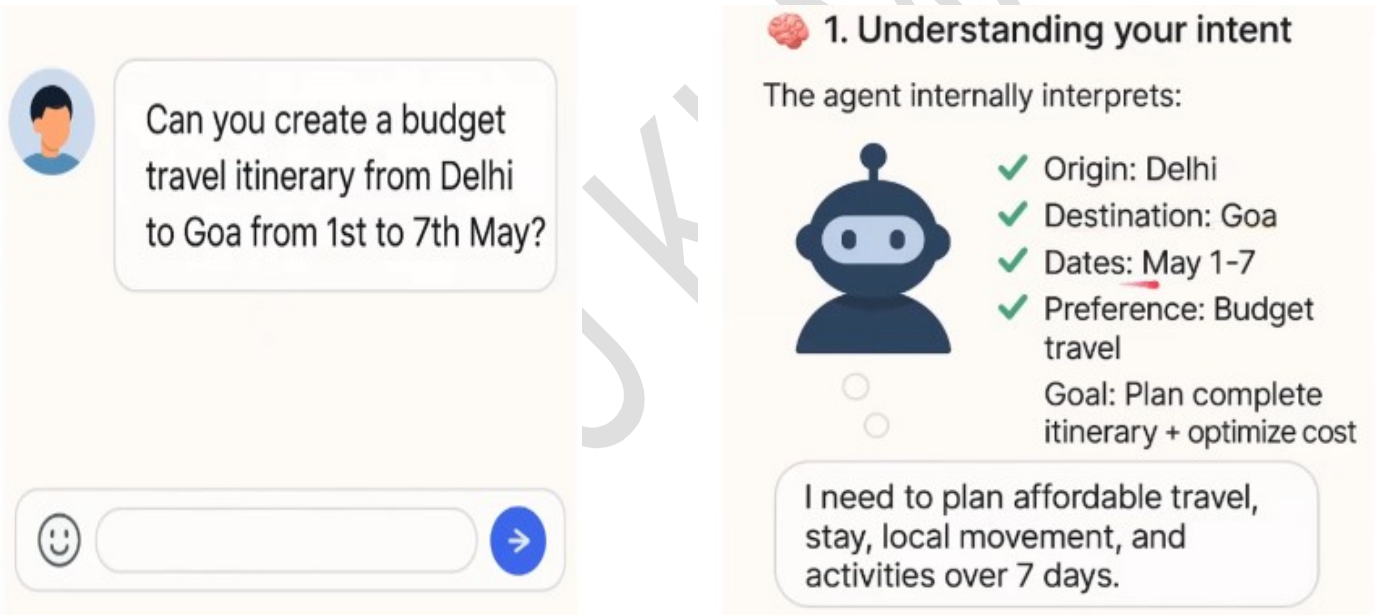
- It can **autonomously plan** multi-step workflows

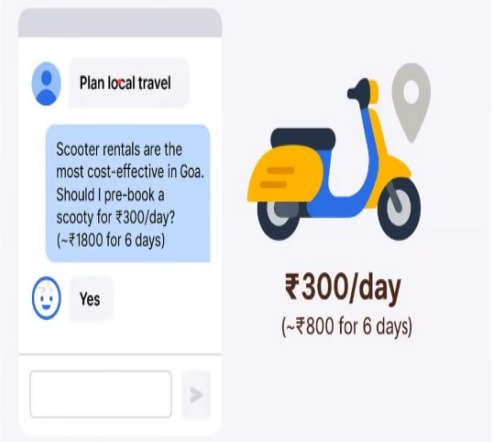
Think of it like this:
A normal LLM = a very smart assistant who can **talk and explain**.
An **Agentic AI** = a smart assistant who can **talk + take actions for you** (bookings, queries, automation).

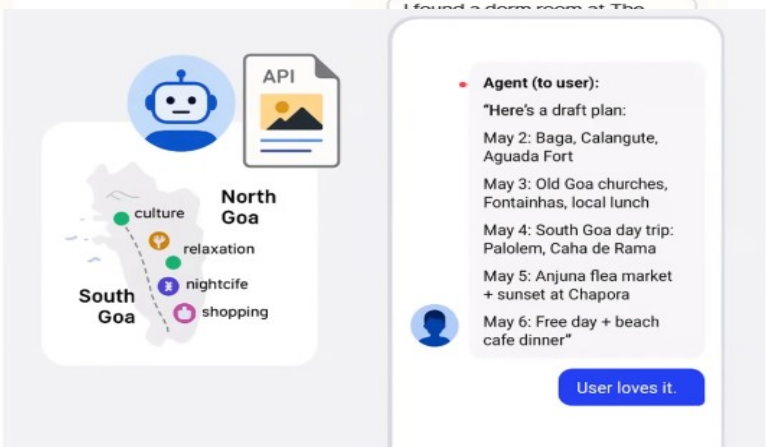
Simple Analogy

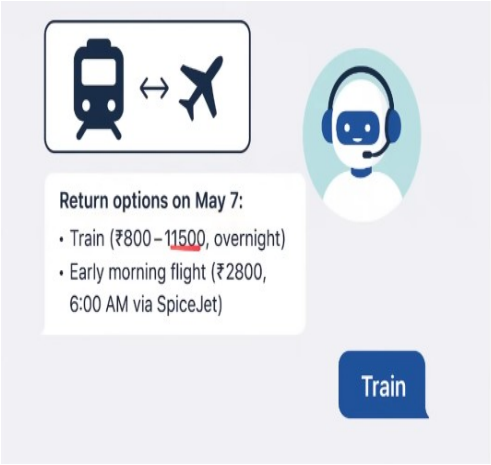
- **LLM only**: “Here’s a list of flights from Pune to New York.” (just information).
- **Agentic AI**: “I booked your flight, reserved a hotel, and arranged a cab. All set!” (information + action).

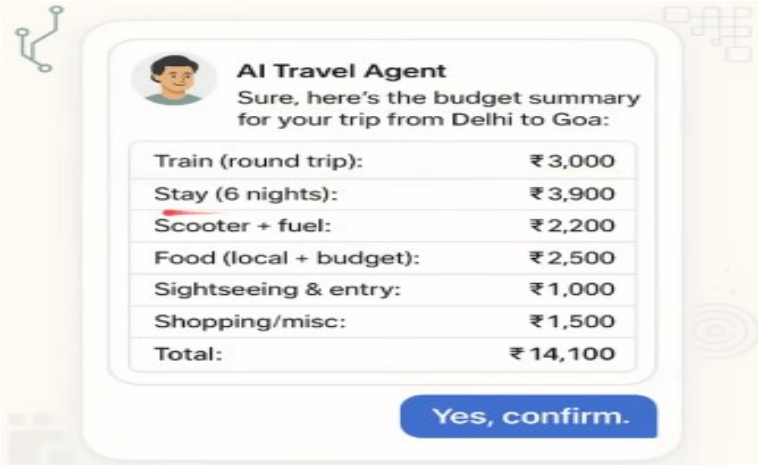
In short:
Agentic AI = LLM + Reasoning + Tools + Actions.
It makes your life easier by **automating tasks end-to-end**, just like a human travel agent would.

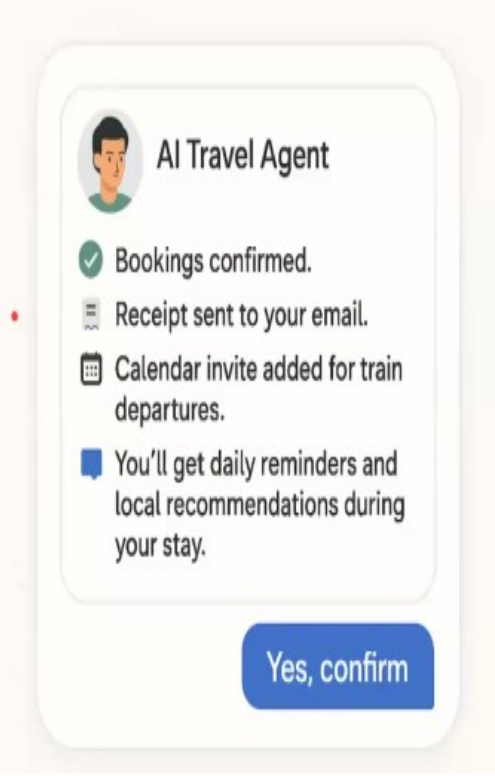






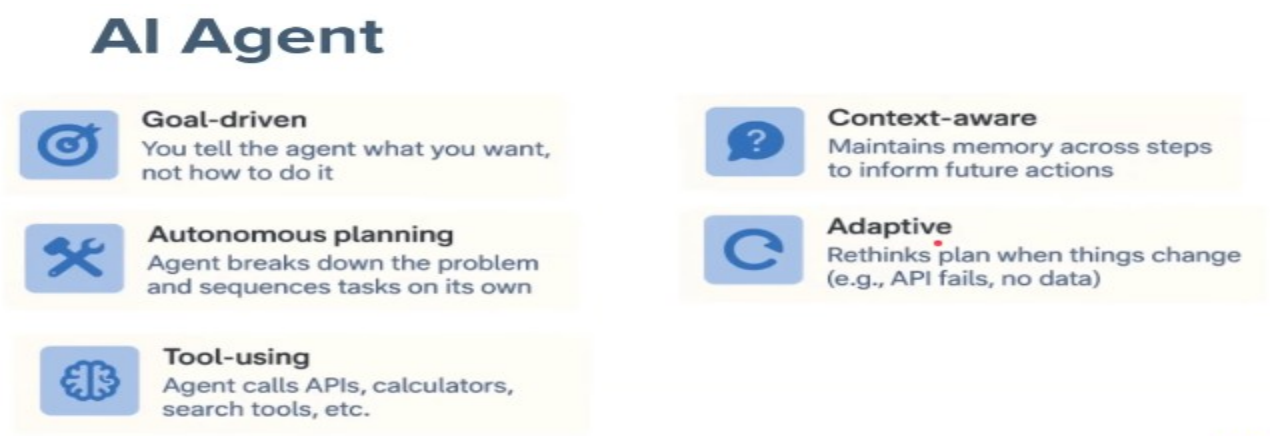






AI Agent

An AI agent is an intelligent system that receives a high-level goal from a user, and autonomously plans, decides, and executes a sequence of actions by using external tools, APIs, or knowledge sources – all while maintaining context, reasoning over multiple steps, adapting to new information, and optimizing for the intended outcome.



1. create_react_agent

Simple:
This function helps you **quickly create an agent** using the **ReAct (Reason + Act) framework**.

- ReAct is a way where the LLM **first reasons** ("what tool should I use?") and then **acts** (calls the tool).
- LangChain gives create_react_agent so you don't have to manually wire reasoning and acting logic.

Technical:

- Takes an LLM + tools + prompt.
- Creates an **agent object** that knows how to "think → choose tool → call tool → observe result → continue".
- Example:

```
from langchain.agents import create_react_agent
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model="gpt-4o-mini")
tools = [multiply_tool, weather_tool]

agent = create_react_agent(llm, tools, prompt="You are a helpful assistant.")
```

Here, the LLM will **reason** (decide if tool needed) → **act** (call tool) → **return answer**.

2. AgentExecutor

Simple:
This is like the **engine** that actually **runs the agent in a loop** until the task is done.

Analogy:

- create_react_agent = designs the **driver's brain** (reasoning).

RISHU KUMAR

- AgentExecutor = gives the **car to the driver** so they can actually drive and reach the destination.

Technical:

- You pass the agent and the tools into it.
- It handles the **loop**: LLM → tool call → get result → feed back → continue → final answer.

```
from langchain.agents import AgentExecutor

executor = AgentExecutor(agent=agent, tools=tools, verbose=True)
result = executor.invoke({"input": "What is 25 * 4?"})
print(result["output"])  # → 100
```

So without AgentExecutor, the agent definition just **sits idle**. This makes it **run step by step**.

3. hub

Simple:
The **LangChain Hub** is like a "GitHub for prompts and agents".

- It's a **central library** of **ready-made prompts, agents, and chains** that you can pull and use instantly.
- Instead of writing prompts manually, you can fetch them.

Technical:

- You can pull prompts/agents from the hub and use them in your project.
- Example:

```
from langchain import hub

# Pull a prebuilt ReAct prompt for agents
prompt = hub.pull("hwchase17/react")
print(prompt)
```

- Developers share high-quality prompts on the hub.
- You can also push your own custom ones.

TL;DR

- **create_react_agent** → makes an agent using ReAct framework (brain setup).
- **AgentExecutor** → actually runs the agent step-by-step (engine).
- **hub** → library of pre-built prompts/agents you can use instead of writing from scratch.

RISHU KUMAR

Agents in LangChain (and generally in LLM systems) can be designed using **different patterns** depending on how you want the LLM to reason, plan, and act.

1. **ReAct Agent (Reason + Act)**

- **Idea:** The LLM *thinks step by step* (reasoning), decides what tool to call, then acts (executes the tool). It is a design pattern used in AI agents that stands for reasoning+acting. It allows a LLM to interleave internal reasoning (Thought) with external actions (like tool use) in a structured, multi-step process.
- Instead of generating an answer in one go, the model thinks step by step, deciding what it needs to do next and optionally calling tools (APIs, calculators, web search, etc.) to help it.
- **Loop:** Reason(thought) → Act → Observe(response/answer) → Repeat.
- **Use case:** Multi-step reasoning like travel planning, research, troubleshooting.
- **Example:**
    - LLM: "To answer the question, I need today's weather."
    - Calls Weather API tool.
    - Uses result + reasoning to answer.

Agent & AgentExecutor:

AgentExecutor orchestrates(runs/handles) the entire loop.

- Sends inputs and previous messages to the agent.
- Gets the next action from agent.
- Executes that tool with provided input.
- Adds the tool's observations back into the history.
- Loops again with updated history until the agent says final answer.

2. Plan-and-Execute Agent

- **Idea:** The LLM first creates a **high-level plan** (like a checklist of tasks).
- Then another executor agent follows the plan step by step.
- **Benefit:** Reduces hallucinations, better for long workflows.
- **Example:** Travel booking:
    - Plan: 1) Find flights, 2) Book hotel, 3) Arrange taxi.
    - Execute each step with tools.

RISHU KUMAR

3. Conversational Agent

- **Idea:** Designed for **chatbots** that can call tools when needed.
- Keeps track of conversation history and context.
- **Example:** Customer support assistant:
  - User: "Show me my last 3 orders."
  - Agent uses DB query tool → retrieves results → responds naturally.

4. Self-Ask-with-Search Agent

- **Idea:** The LLM breaks a complex query into **sub-questions**, and uses search tools to answer them.
- **Example:** "Who is the current president of France's wife?"
  - Step 1: "Who is current president of France?" → Search → Emmanuel Macron.
  - Step 2: "Who is Emmanuel Macron's wife?" → Search → Brigitte Macron.

5. MRKL Agent (Modular Reasoning, Knowledge, and Language)

- **Idea:** A **routing agent** that decides which expert tool/module should solve the query.
- **Example:**
  - Math question → send to Calculator tool.
  - Wikipedia question → send to Wiki retriever.
  - Code task → send to Python REPL.

6. Multi-Action Agent

- **Idea:** Can **call multiple tools at once** in parallel.
- Useful when tasks are independent.
- **Example:** Ask: "Give me today's weather in Paris and New York."
  - Calls Weather API twice (parallel), combines result.

7. Memory-Enhanced Agents

- **Idea:** Agent stores memory of past interactions and uses it to decide.
- **Use case:** Personal assistants, where user preferences are remembered.

In LangChain, you can build these with helpers like:

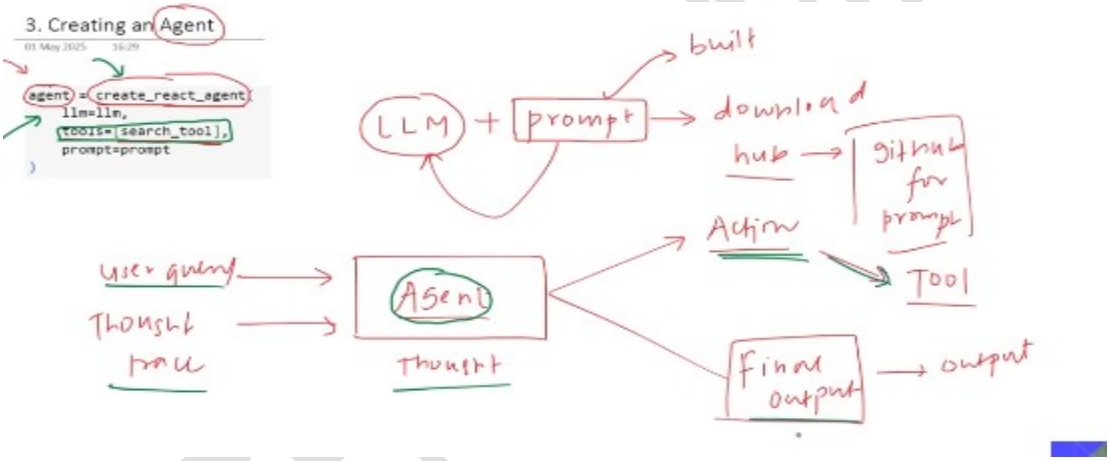
- create_react_agent → for ReAct style.

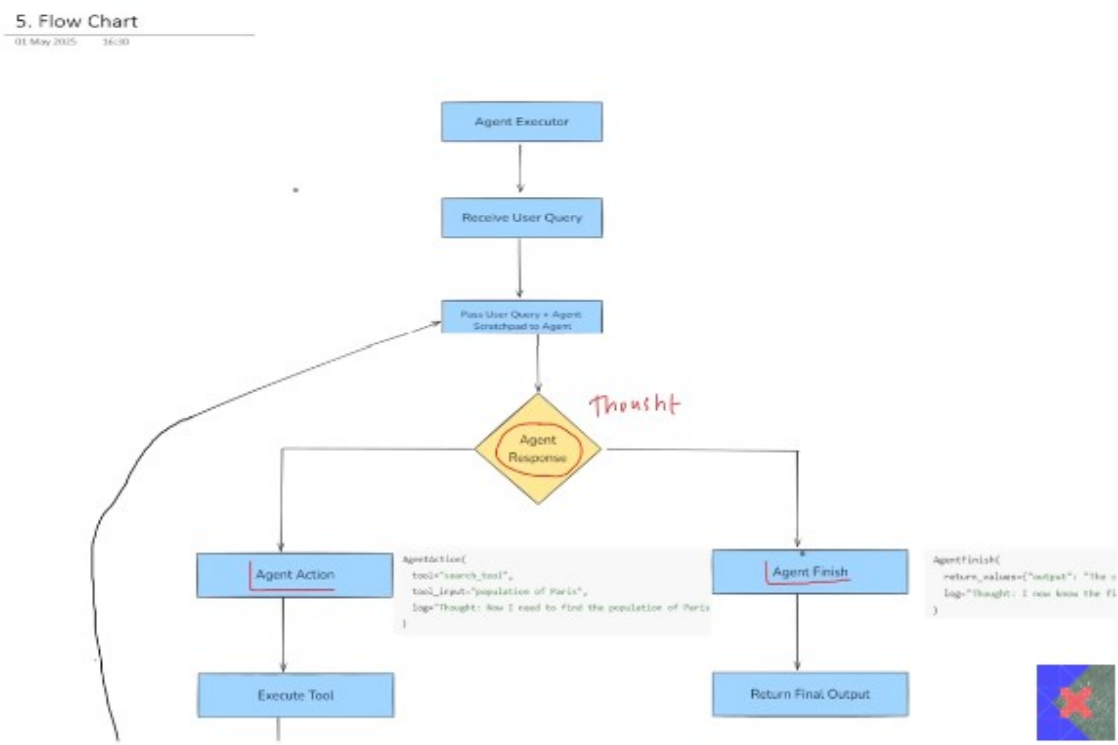
RISHU KUMAR

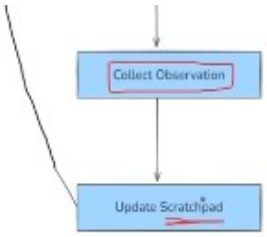
- PlanAndExecute → for planner-executor pattern.
- initialize_agent with different configurations.

TL;DR:
Agents = LLMs + tools + reasoning loop.

- **ReAct** → reason+act repeatedly.
- **Plan-and-Execute** → make plan then run it.
- **Conversational** → tool-using chatbot.
- **Self-Ask** → break queries into sub-questions.
- **MRKL** → route to expert tools.
- **Multi-Action** → call tools in parallel.
- **Memory-based** → remember user history.







Now, making AI agent using langchain is an old way. And if we want to make an industry grade AI agent then we don’t use langchain. Currently it is possible to make AI agent using langchain but it’s not recommended. So to make scalable AI agent, we should use LangGraph.

RISHU KUMAR

THANK YOU!!!